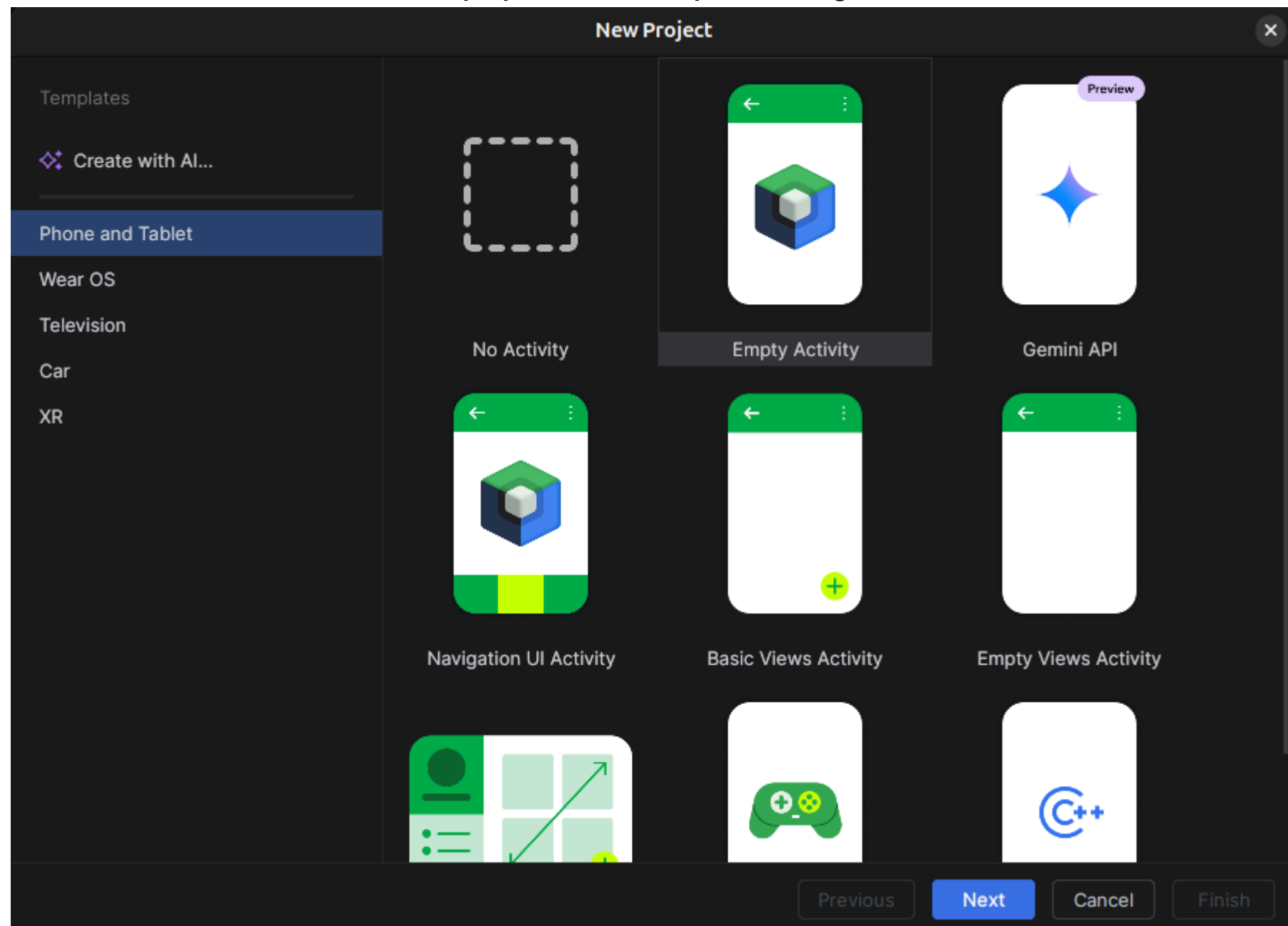


Crear mi propia herramienta para descargar videos



New Project

×

Empty Activity

Create a new empty activity with Jetpack Compose

Name

My Application

Package name

com.baycodec.myapplication

Save location

/home/baycodec/AndroidStudioProjects/MyApplication

Minimum SDK

API 24 ("Nougat"; Android 7.0)

ⓘ

Your app will run on approximately 99,2% of devices.

[Help me choose](#)

Build configuration language ⓘ

Kotlin DSL (build.gradle.kts) [Recommended]

Previous

Next

Cancel

Finish

Configuración de la pantalla actual

- **Name:** Puedes ponerle algo como `VideoDownloaderApp`.
- **Package name:** Esto identifica tu aplicación de forma única. `com.ejemplo.videodownloader` está bien.
- **Minimum SDK: API 24 (Nougat)** es una excelente elección; cubre la gran mayoría de dispositivos actuales y es compatible con las librerías modernas de descarga.
- **Build configuration language:** Mantén **Kotlin DSL (build.gradle.kts)**, ya que es el estándar actual y facilita la integración de librerías modernas.

New Project

Empty Activity

Create a new empty activity with Jetpack Compose

Name

VideoDownloaderApp

Package name

com.baycodec.videodownloaderapp

Save location

/home/baycodec/AndroidStudioProjects/VideoDownloaderApp

Minimum SDK

API 24 ("Nougat"; Android 7.0)

Your app will run on approximately 99,2% of devices.

[Help me choose](#)

Build configuration language ?

Kotlin DSL (build.gradle.kts) [Recommended]

Previous

Next

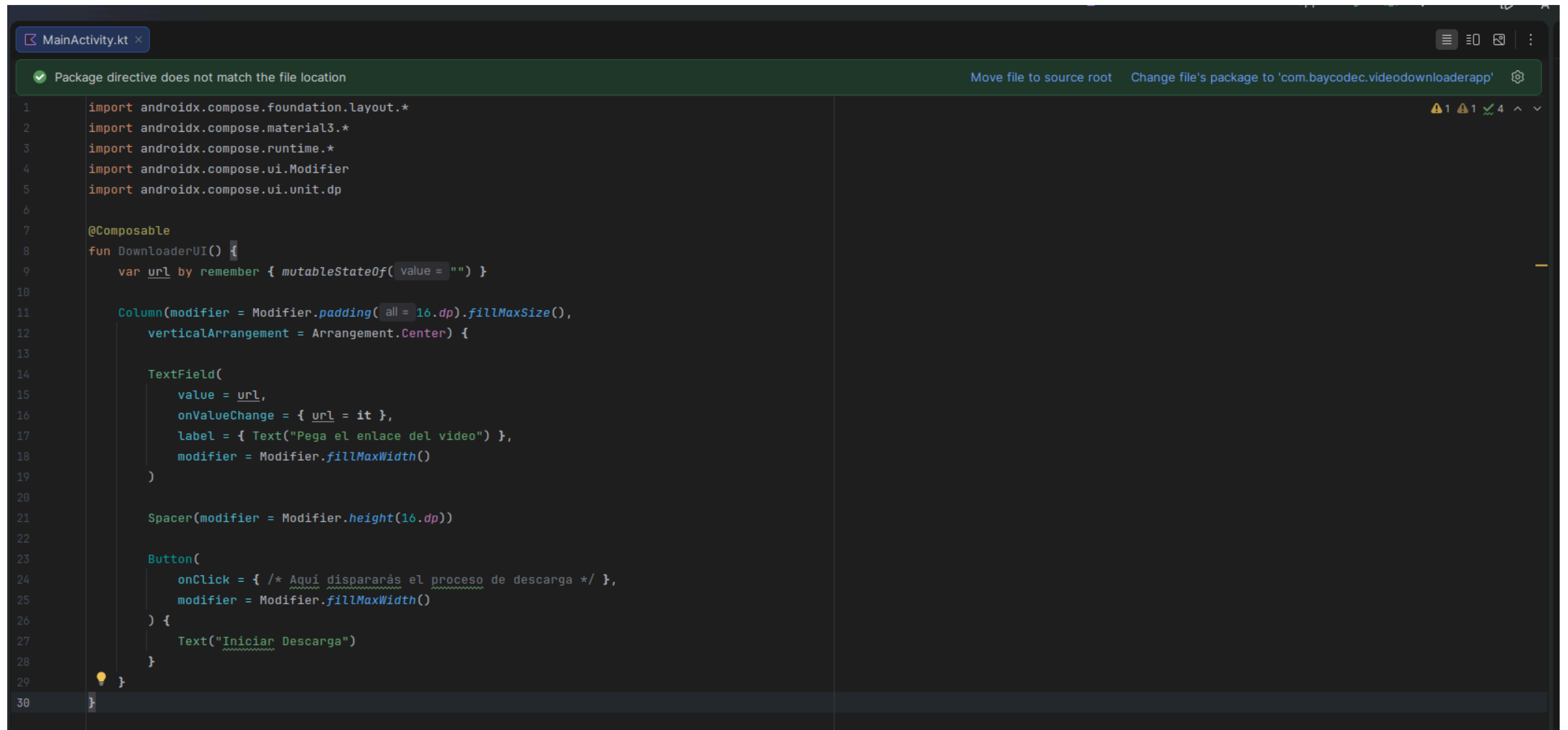
Cancel

Finish

```
1 package com.baycode.videodownloaderapp
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.padding
9 import androidx.compose.material3.Scaffold
10 import androidx.compose.material3.Text
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.tooling.preview.Preview
14 import com.baycode.videodownloaderapp.ui.theme.VideoDownloaderAppTheme
15
16 class MainActivity : ComponentActivity() {
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         enableEdgeToEdge()
20         setContent {
21             VideoDownloaderAppTheme {
22                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
23                     Greeting(
24                         name = "Android",
25                         modifier = Modifier.padding(innerPadding)
26                     )
27                 }
28             }
29         }
30     }
31 }
32
33 @Composable
34 fun Greeting(name: String, modifier: Modifier = Modifier) {
35     Text(
36         text = "Hello $name!",
37         modifier = modifier
38     )
39 }
40
41 @Preview(showBackground = true)
42 @Composable
43 fun GreetingPreview() {
44     VideoDownloaderAppTheme {
45         Greeting("Android")
46     }
47 }
48
```


State Management (Gestión de Estado): Fíjate en el uso de `remember` y `mutableStateOf`. En Compose, esto es lo que permite que la pantalla se actualice cuando el usuario escribe. 2. **Coroutines (Programación Asíncrona):** Como la descarga no es instantánea, si la ejecutas en el hilo principal, la app se bloqueará ("ANR" o *Application Not Responding*). Usarás `CoroutineScope` para mover la tarea de descarga a un hilo secundario (Background).

Permisos en Tiempo de Ejecución (Runtime Permissions): A partir de Android 6.0, no basta con poner los permisos en el `Manifest`. Tu código debe solicitar el permiso de escritura al usuario mediante una ventana emergente la primera vez que intente descargar algo.



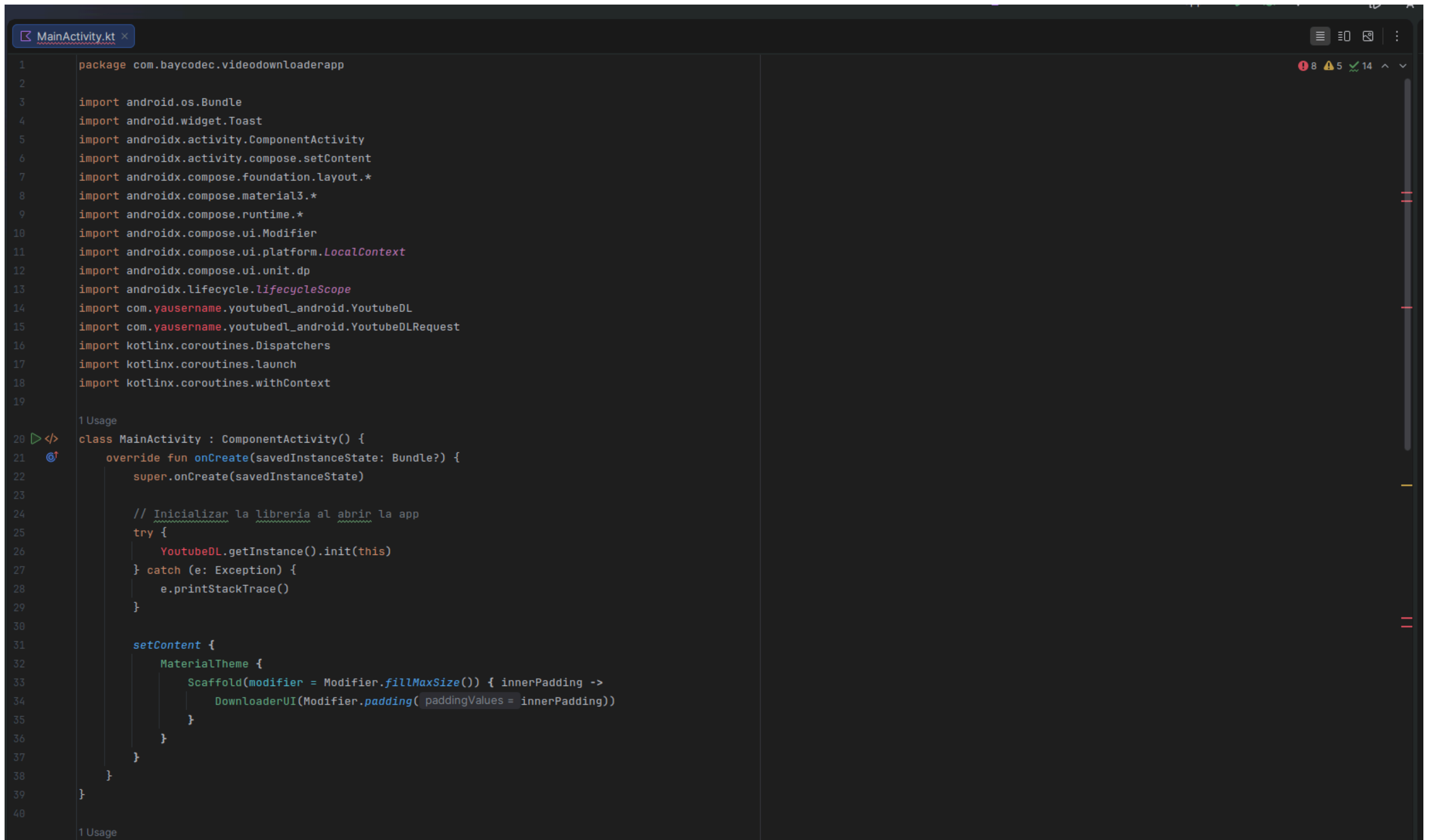
```
1  import androidx.compose.foundation.layout.*
2  import androidx.compose.material3.*
3  import androidx.compose.runtime.*
4  import androidx.compose.ui.Modifier
5  import androidx.compose.ui.unit.dp
6
7  @Composable
8  fun DownloaderUI() {
9      var url by remember { mutableStateOf(value = "") }
10
11      Column(modifier = Modifier.padding(all = 16.dp).fillMaxSize(),
12              verticalArrangement = Arrangement.Center) {
13
14          TextField(
15              value = url,
16              onValueChange = { url = it },
17              label = { Text("Pega el enlace del video") },
18              modifier = Modifier.fillMaxWidth()
19          )
20
21          Spacer(modifier = Modifier.height(16.dp))
22
23          Button(
24              onClick = { /* Aquí dispararás el proceso de descarga */ },
25              modifier = Modifier.fillMaxWidth()
26          ) {
27              Text("Iniciar Descarga")
28          }
29      }
30 }
```

Añadir el repositorio de yt-dlp-android: Debes ir a tu archivo `build.gradle.kts` (el del módulo `:app`) y añadir la librería. Esto permite que tu app "entienda" cómo comunicarse con el binario de descarga. **Gestión de Permisos:** Necesitas solicitar permisos de acceso a archivos. Investiga sobre la clase `ActivityResultContracts.RequestPermission()` en Android, que es la forma moderna de pedir permisos al usuario en tiempo de ejecución.

Coroutines (Programación asíncrona): Cuando el usuario presione el botón, debes lanzar una **Corrutina** (`lifecycleScope.launch`) para que el proceso ocurra en segundo plano. Si lo haces directamente, la pantalla se congelará y Android cerrará la aplicación por seguridad.

Agregar la dependencia (Tu "motor" de descarga)

Debes abrir el archivo `build.gradle.kts` (Module :app). En la sección de `dependencies`, debes añadir la librería que hace posible la descarga (esta es la librería que maneja `yt-dlp` en Android):



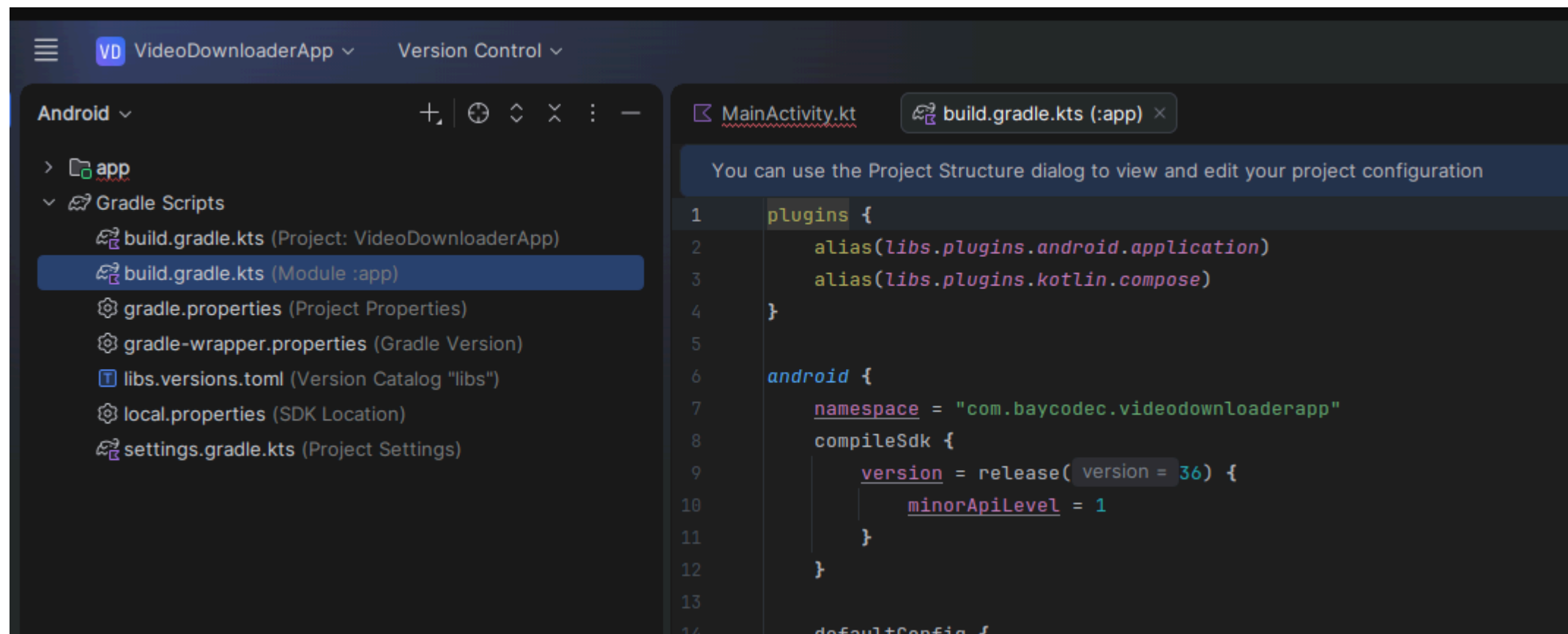
```
1 package com.baycode.videodownloaderapp
2
3 import android.os.Bundle
4 import android.widget.Toast
5 import androidx.activity.ComponentActivity
6 import androidx.activity.compose.setContent
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.platform.LocalContext
12 import androidx.compose.ui.unit.dp
13 import androidx.lifecycle.LifecycleScope
14 import com.yausername.youtubedl_android.YoutubeDL
15 import com.yausername.youtubedl_android.YoutubeDLRequest
16 import kotlinx.coroutines.Dispatchers
17 import kotlinx.coroutines.launch
18 import kotlinx.coroutines.withContext
19
20 1 Usage
21 class MainActivity : ComponentActivity() {
22     override fun onCreate(savedInstanceState: Bundle?) {
23         super.onCreate(savedInstanceState)
24
25         // Inicializar la librería al abrir la app
26         try {
27             YoutubeDL.getInstance().init(this)
28         } catch (e: Exception) {
29             e.printStackTrace()
30         }
31
32         setContent {
33             MaterialTheme {
34                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
35                     DownloaderUI(Modifier.padding(paddingValues = innerPadding))
36                 }
37             }
38         }
39     }
40
41 1 Usage
```

Configuración necesaria (Gradle)

Como mencionamos, esto es vital para que el código anterior compile.

En **settings.gradle.kts**: Asegúrate de tener el repositorio de JitPack:

```
MainActivity.kt
1  dependencyResolutionManagement {
2      repositories {
3          google()
4          mavenCentral()
5          maven { url = uri("https://jitpack.io") }
6      }
7  }
```



```
MainActivity.kt
1 dependencies {
2     // ... tus otras dependencias existentes
3     implementation("com.github.yausername.youtubedl-android:library:0.12.0")
4     implementation("com.github.yausername.youtubedl-android:ffmpeg:0.12.0")
5 }
6
```

Permisos: No olvides ir a tu archivo `AndroidManifest.xml` y añadir `<uses-permission android:name="android.permission.INTERNET" />` dentro de la etiqueta `<manifest>` (antes de `<application>`).

Contexto Académico: Este código es una implementación base. Para un proyecto universitario de mayor nivel, te recomiendo implementar `RequestPermission` para que la app pida acceso al almacenamiento de forma profesional, en lugar de solo imprimir un `Toast`.

Ejecución: Al compilar, asegúrate de tener una conexión estable, ya que la primera vez la librería descargará el binario de `yt-dlp` necesario para funcionar.

Android

app

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

```
5 import androidx.activity.ComponentActivity
6 import androidx.activity.compose.setContent
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.platform.LocalContext
12 import androidx.compose.ui.unit.dp
13 import androidx.lifecycle.LifecycleScope
14 import com.yausername.youtubedl_android.YoutubeDL
15 import com.yausername.youtubedl_android.YoutubeDLRequest
16 import kotlinx.coroutines.Dispatchers
17 import kotlinx.coroutines.launch
18 import kotlinx.coroutines.withContext
19
20 1 Usage
21 class MainActivity : ComponentActivity() {
22     override fun onCreate(savedInstanceState: Bundle?) {
23         super.onCreate(savedInstanceState)
24
25         // Inicializar la libreria al abrir la app
26         try {
27             YoutubeDL.getInstance().init(this)
28         } catch (e: Exception) {
29             e.printStackTrace()
30         }
31     }
32 }
```

Gradle

VideoDownloaderApp

Tasks

app

Build

Sync

Build Output

> :app:checkDebugAarMetadata 1 error 536 ms

> :app:compileDebugKotlin 8 errors 6 sec, 788 ms

▼ MainActivity.kt app/src/main/java/com/baycodec/videodownloaderapp 8 errors

Unresolved reference 'yausername'. :14

Unresolved reference 'yausername'. :15

Unresolved reference 'YoutubeDL'. :26

Unresolved reference 'YoutubeDLRequest'. :60

Unresolved reference 'YoutubeDL'. :61

Cannot infer type for this parameter. Specify it explicitly. :61

Cannot infer type for this parameter. Specify it explicitly. :61

Cannot infer type for this parameter. Specify it explicitly. :61

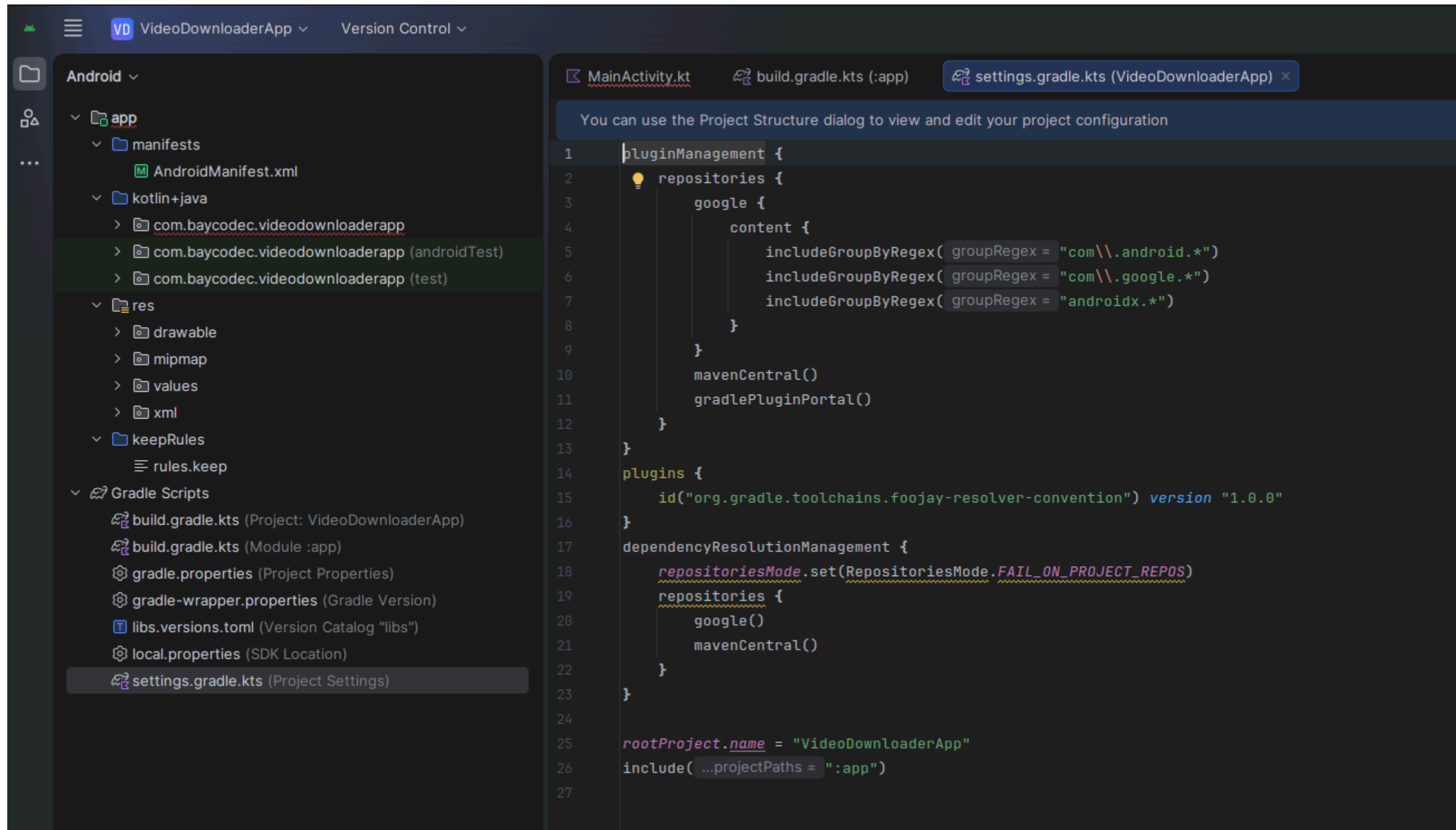
3 issues were found when checking AAR metadata:

e: file:///home/baycodec/AndroidStudioProjects/VideoDownloaderApp/app/src/main/java/com/baycodec/videodownloaderapp/MainActivity.kt:14:12 Unresolved reference 'yausername'.

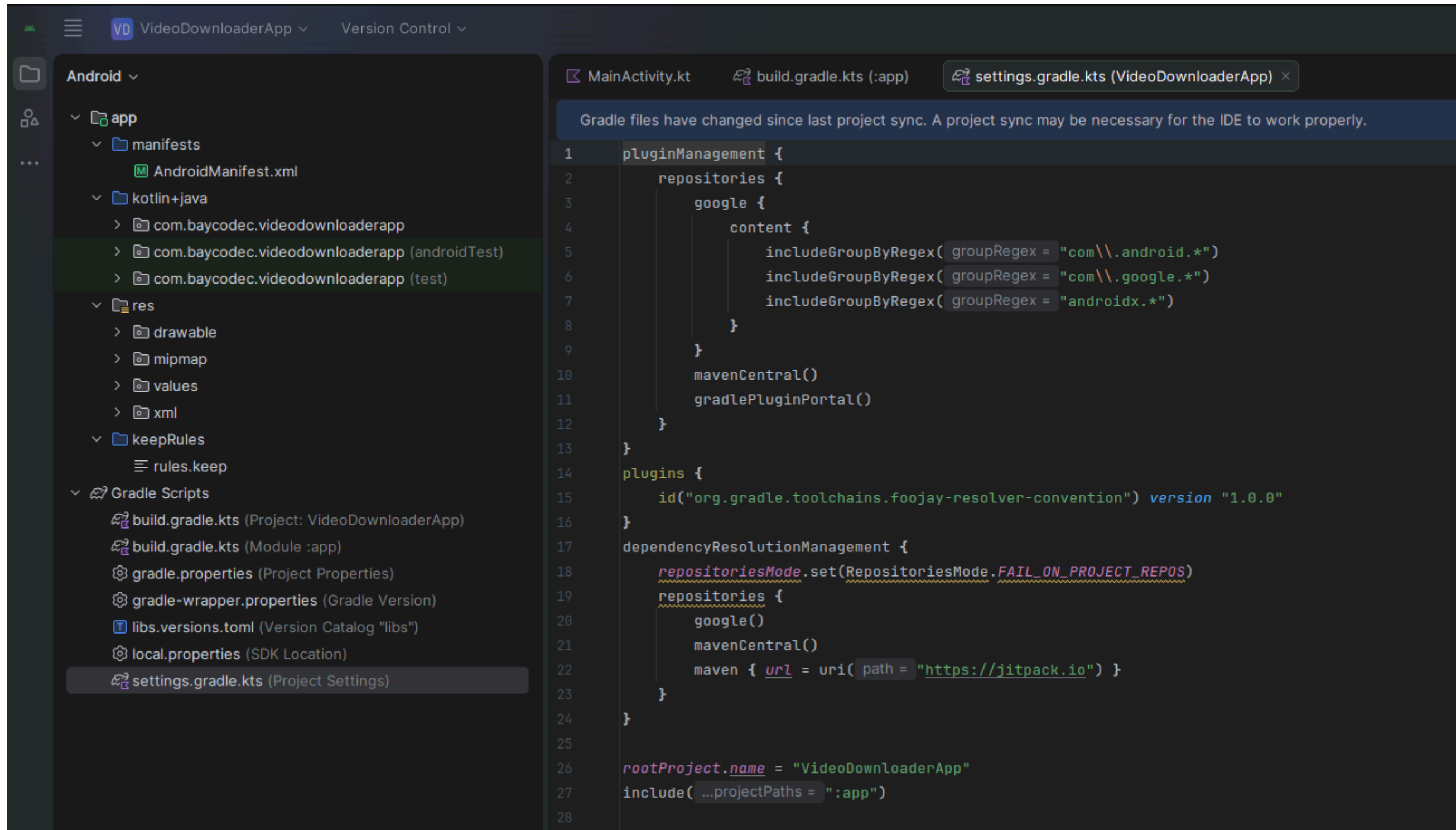
Fix with AI

VideoDownloaderApp > app > src > main > java > com > baycodec > videodownloaderapp > MainActivity.kt

14:12 LF UTF-8 4 spaces



La librería youtubedl-android está en JitPack, que no está en los repositorios del proyecto.
agregar JitPack al settings.gradle.kts, la versión al catálogo TOML,
y la dependencia al build.gradle.kts.



1 Paso 2 – Agregar la versión y la librería al catálogo:

2
3 ● Update(gradle/libs.versions.toml)

4
5
6 Edit file

7 gradle/libs.versions.toml

8
9 8 activityCompose = "1.13.0"

10 9 kotlin = "2.2.10"

11 10 composeBom = "2026.02.01"

12 11 +youtubedlAndroid = "0.17.2"

13 12

14 13 [libraries]

15 14 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }

16

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

[versions]

agp = "9.2.1"

coreKtx = "1.19.0"

junit = "4.13.2"

junitVersion = "1.3.0"

espressoCore = "3.7.0"

lifecycleRuntimeKtx = "2.11.0"

activityCompose = "1.13.0"

kotlin = "2.2.10"

composeBom = "2026.02.01"

[libraries]

androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }

junit = { group = "junit", name = "junit", version.ref = "junit" }

androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }

androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }

androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }

androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }

androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }

androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }

androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }

androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }

androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }

androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }

androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }

androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }

[plugins]

android-application = { id = "com.android.application", version.ref = "agp" }

kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

1

[versions]

2

agp = "9.2.1"

3

coreKtx = "1.19.0"

4

junit = "4.13.2"

5

junitVersion = "1.3.0"

6

espressoCore = "3.7.0"

7

lifecycleRuntimeKtx = "2.11.0"

8

activityCompose = "1.13.0"

9

kotlin = "2.2.10"

10

composeBom = "2026.02.01"

11

12

[libraries]

13

androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }

14

junit = { group = "junit", name = "junit", version.ref = "junit" }

15

androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }

16

androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }

17

androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }

18

androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }

19

androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }

20

androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }

21

androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }

22

androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }

23

androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }

24

androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }

25

androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }

26

androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }

27

28

[plugins]

29

android-application = { id = "com.android.application", version.ref = "agp" }

30

kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }

31

32

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

1

[versions]

2

agp = "9.2.1"

3

coreKtx = "1.19.0"

4

junit = "4.13.2"

5

junitVersion = "1.3.0"

6

espressoCore = "3.7.0"

7

lifecycleRuntimeKtx = "2.11.0"

8

activityCompose = "1.13.0"

9

kotlin = "2.2.10"

10

composeBom = "2026.02.01"

11

youtubedlAndroid = "0.17.2"

12

13

[libraries]

14

androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }

15

junit = { group = "junit", name = "junit", version.ref = "junit" }

16

androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }

17

androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }

18

androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }

19

androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }

20

androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }

21

androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }

22

androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }

23

androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }

24

androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }

25

androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }

26

androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }

27

androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }

28

29

[plugins]

30

android-application = { id = "com.android.application", version.ref = "agp" }

31

kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }

32

33

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

1

[versions]

2

agp = "9.2.1"

3

coreKtx = "1.19.0"

4

junit = "4.13.2"

5

junitVersion = "1.3.0"

6

espressoCore = "3.7.0"

7

lifecycleRuntimeKtx = "2.11.0"

8

activityCompose = "1.13.0"

9

kotlin = "2.2.10"

10

composeBom = "2026.02.01"

11

yotubedlAndroid = "0.17.2"

12

13

[libraries]

14

androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }

15

junit = { group = "junit", name = "junit", version.ref = "junit" }

16

androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }

17

androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }

18

androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }

19

androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }

20

androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }

21

androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }

22

androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }

23

androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }

24

androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }

25

androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }

26

androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }

27

androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }

28

29

[plugins]

30

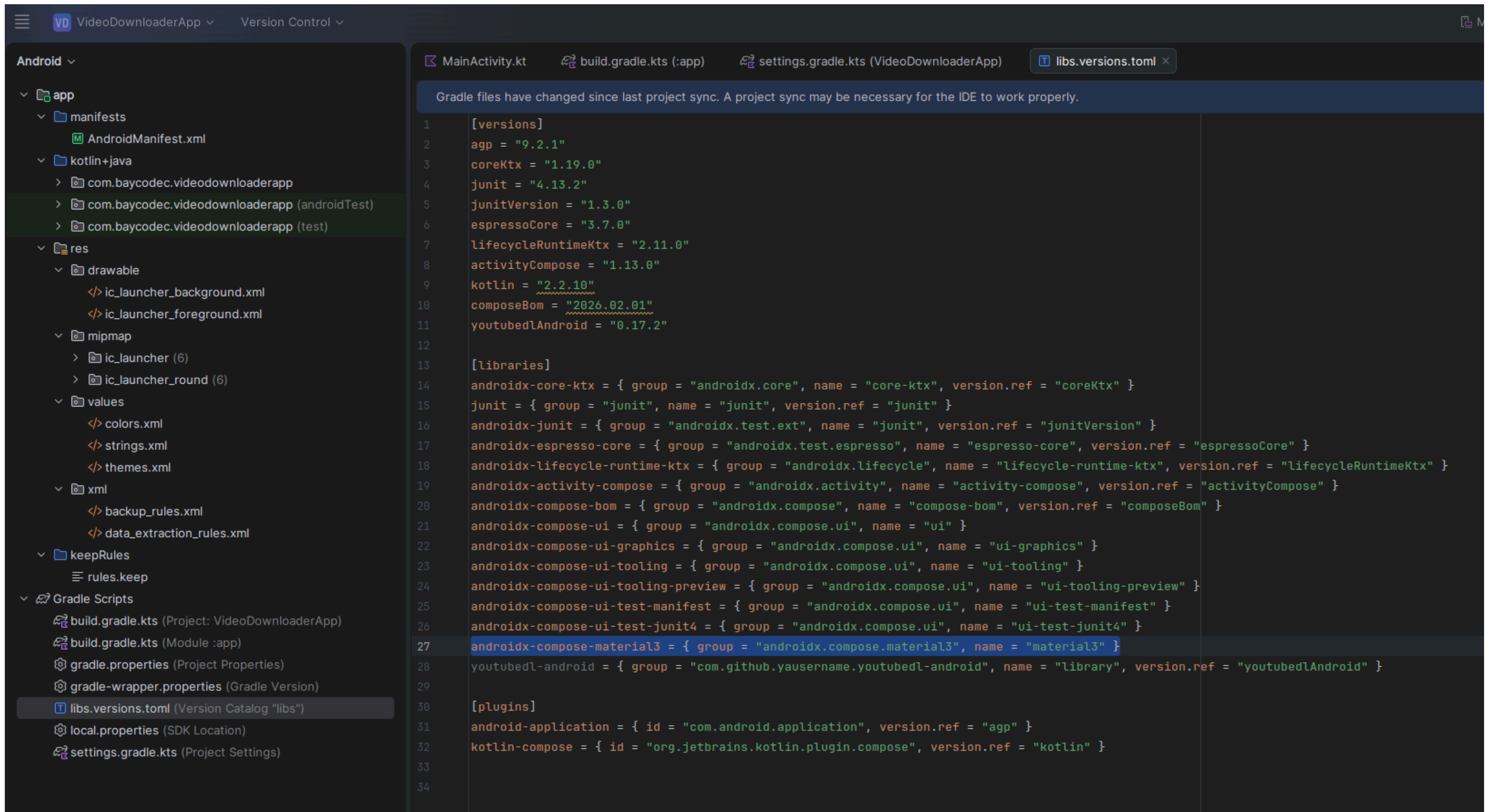
android-application = { id = "com.android.application", version.ref = "agp" }

31

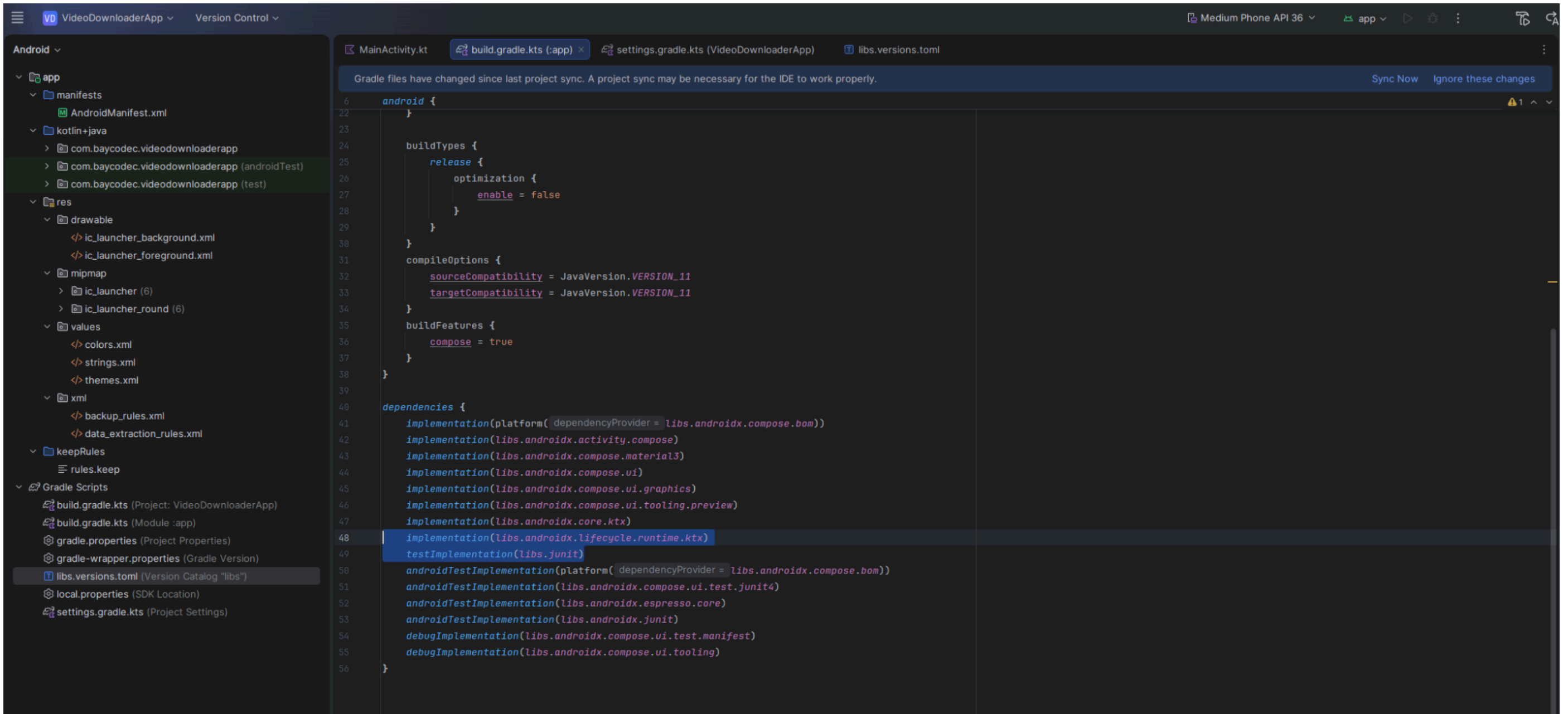
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }

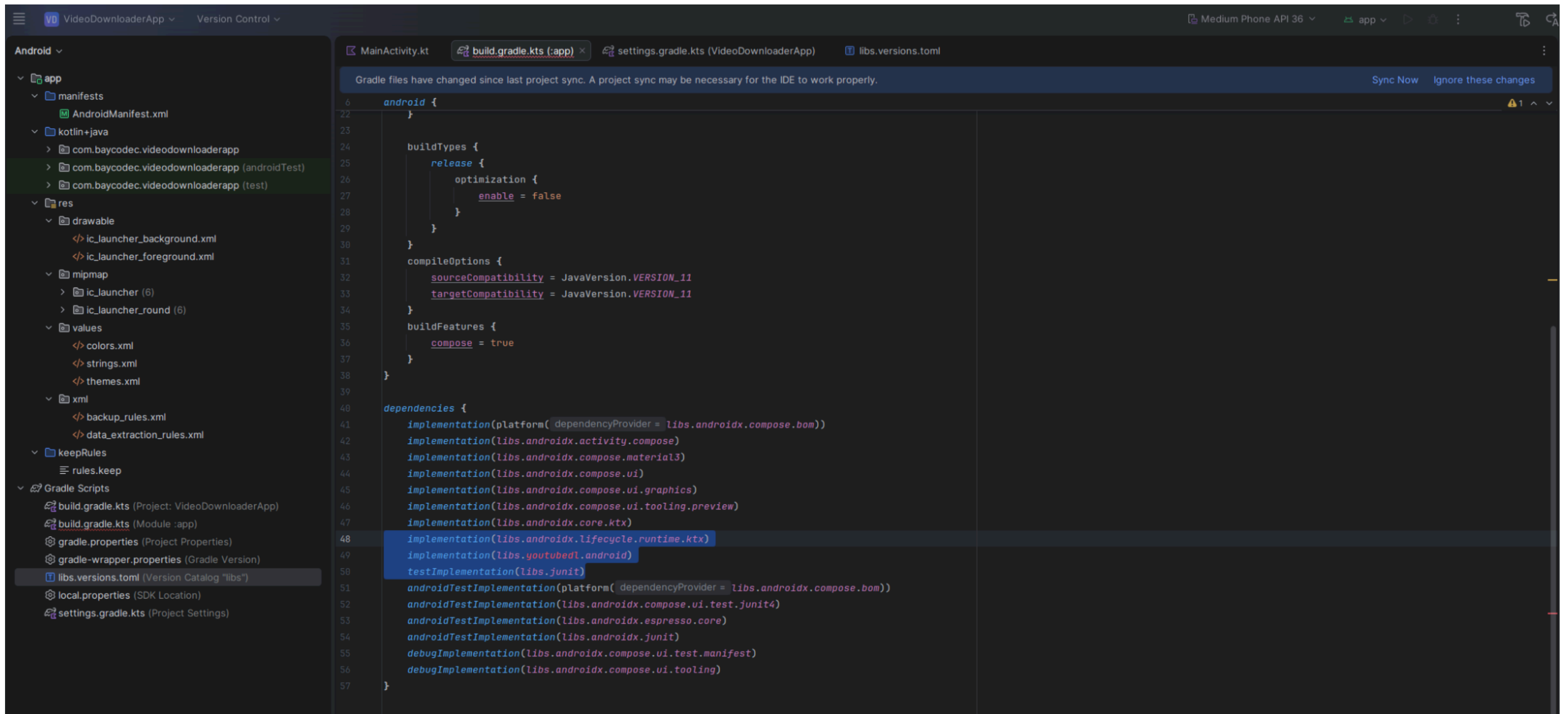
32

33



Paso 3 — Agregar la dependencia en build.gradle.kts:



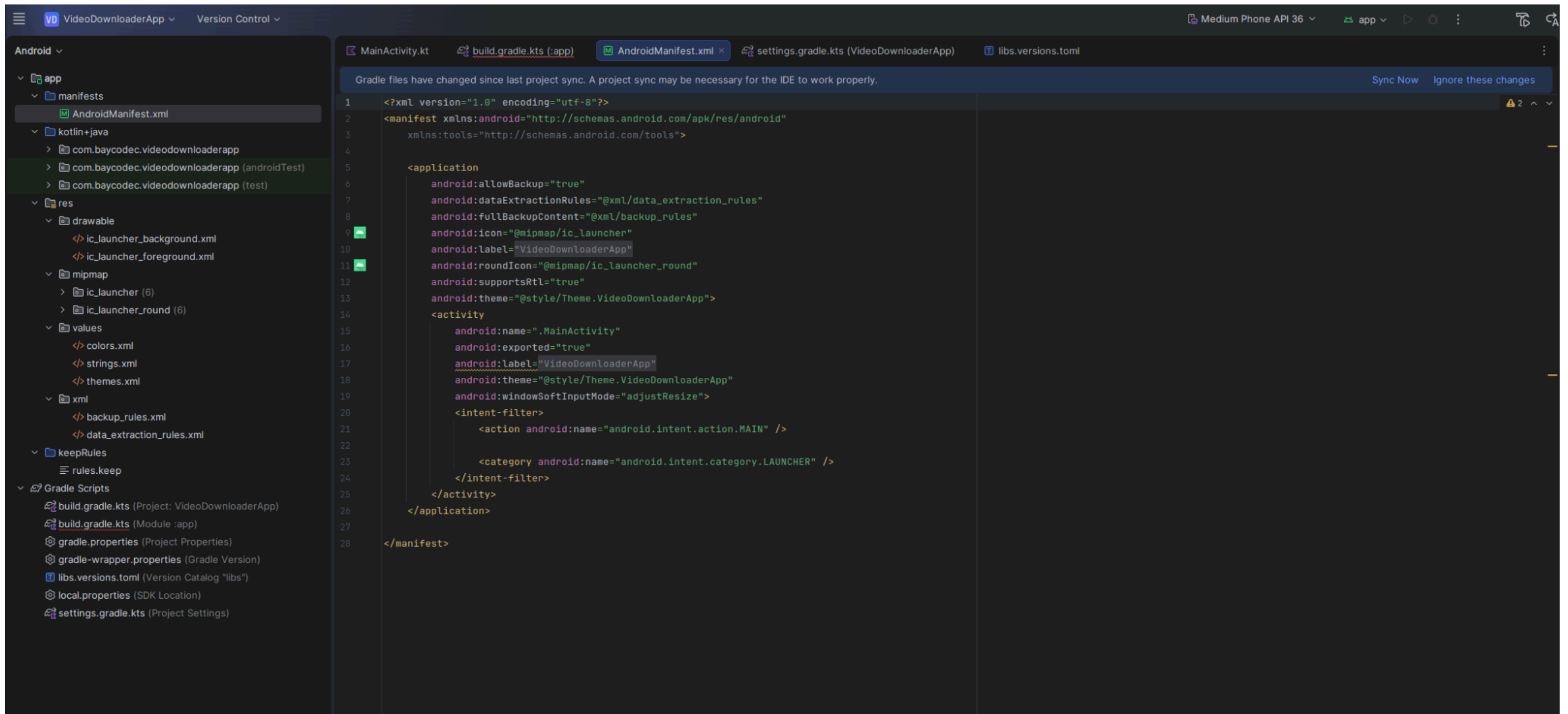


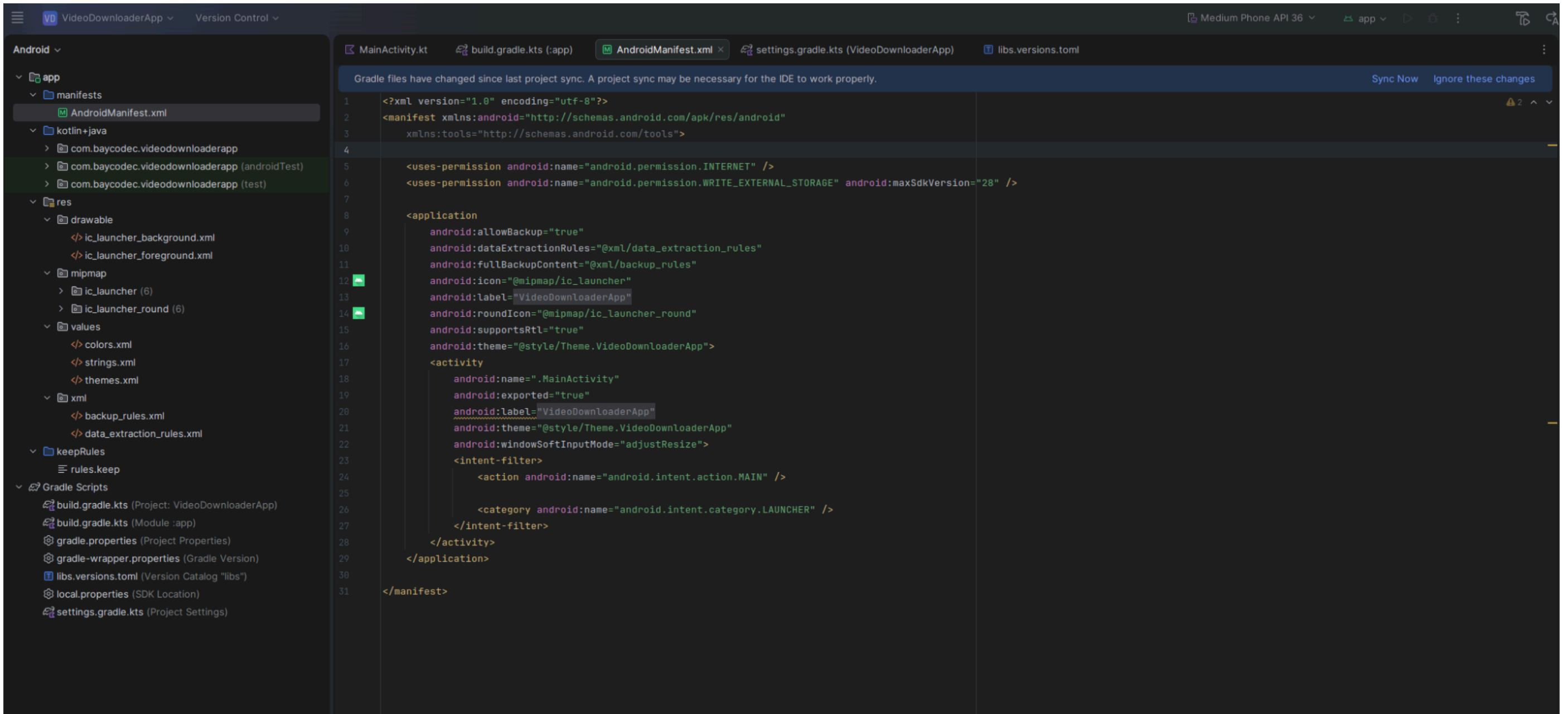
Hasta este punto ya solucionas la primera parte que es las dependencia ahora solo te falta por cumplir permisos y el proceso de descarga tranquilo tu puedes

Nivel a cumplir

- 2 Permisos INTERNET y WRITE_EXTERNAL_STORAGE ausentes en el Manifest - La app crashea al intentar descargar
- 3 La app crashea al intentar descargar La app crashea al intentar descargar
- 4 Callback de progreso vacío El usuario ve la UI congelada sin feedback
- 5 Botón no se deshabilita durante la descarga - El usuario puede pulsar varias veces y lanzar descargas en paralelo

app/src/main/AndroidManifest.xml





MainActivity.kt build.gradle.kts (:app) AndroidManifest.xml × settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3         xmlns:tools="http://schemas.android.com/tools">
4
5      <uses-permission android:name="android.permission.INTERNET" />
6      <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" android:maxSdkVersion="28" />
7
8      <application
9          android:allowBackup="true"
10         android:dataExtractionRules="@xml/data_extraction_rules"
11         android:fullBackupContent="@xml/backup_rules"
12         android:icon="@mipmap/ic_launcher"
13         android:label="VideoDownloaderApp"
14         android:roundIcon="@mipmap/ic_launcher_round"
15         android:supportsRtl="true"
16         android:theme="@style/Theme.VideoDownloaderApp">
17          <activity
18              android:name=".MainActivity"
19              android:exported="true"
20              android:label="VideoDownloaderApp"
21              android:theme="@style/Theme.VideoDownloaderApp"
22              android:windowSoftInputMode="adjustResize">
23              <intent-filter>
24                  <action android:name="android.intent.action.MAIN" />
25
26                  <category android:name="android.intent.category.LAUNCHER" />
27              </intent-filter>
28          </activity>
29      </application>
30
31  </manifest>
```

INTERNET

Sin este permiso Android bloquea cualquier conexión de red a nivel de sistema operativo. youtubedl-android necesita conectarse al servidor para obtener la URL real del video y descargarlo. Sin esto la app crashea con `NetworkOnMainThreadException` o `SecurityException` antes de hacer nada.

WRITE_EXTERNAL_STORAGE

Permite escribir archivos en el almacenamiento externo (la tarjeta SD o la carpeta compartida del dispositivo). Solo lo necesitan Android 9 y versiones anteriores.

`android:maxSdkVersion="28"`

Esto le dice al sistema que el permiso solo aplica hasta Android 9 (API 28). Desde Android 10 (API 29) Google cambió el modelo de almacenamiento a Scoped Storage: las apps ya no pueden escribir libremente en cualquier carpeta, sino solo en sus propios directorios o a través de MediaStore. Si no ponemos este límite, en Android 10+ el sistema igual ignora el permiso, pero lo declaramos sin sentido. El acceso real en Android 10+ lo manejaremos en el Error 3 cuando definamos la ruta de descarga correcta.

MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
1 package com.baycode.videodownloaderapp
2
3 import android.os.Bundle
4 import android.widget.Toast
5 import androidx.activity.ComponentActivity
6 import androidx.activity.compose.setContent
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.platform.LocalContext
12 import androidx.compose.ui.unit.dp
13 import androidx.lifecycle.LifecycleScope
14 import com.yausername.youtubedl_android.YoutubeDL
15 import com.yausername.youtubedl_android.YoutubeDLRequest
16 import kotlinx.coroutines.Dispatchers
17 import kotlinx.coroutines.launch
18 import kotlinx.coroutines.withContext
19
20 class MainActivity : ComponentActivity() {
21     override fun onCreate(savedInstanceState: Bundle?) {
22         super.onCreate(savedInstanceState)
23
24         // Inicializar la librería al abrir la app
25         try {
26             YoutubeDL.getInstance().init(this)
27         } catch (e: Exception) {
28             e.printStackTrace()
29         }
30
31         setContent {
32             MaterialTheme {
```

El problema es que YoutubeDLRequest(url) no sabe dónde guardar el archivo, así que lo deja en un directorio temporal o falla. Hay que indicarle la ruta de salida con la opción -o de yt-dlp.

MainActivity.kt x build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
1 package com.baycode.videodownloaderapp
2
3 import android.os.Bundle
4 import android.os.Environment
5 import android.widget.Toast
6 import androidx.activity.ComponentActivity
7 import androidx.activity.compose.setContent
8 import androidx.compose.foundation.layout.*
9 import androidx.compose.material3.*
10 import androidx.compose.runtime.*
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.platform.LocalContext
13 import androidx.compose.ui.unit.dp
14 import androidx.lifecycle.LifecycleScope
15 import com.yausername.youtubedl_android.YoutubeDL
16 import com.yausername.youtubedl_android.YoutubeDLRequest
17 import kotlinx.coroutines.Dispatchers
18 import kotlinx.coroutines.launch
19 import kotlinx.coroutines.withContext
20
21 1 Usage
22 class MainActivity : ComponentActivity() {
23     override fun onCreate(savedInstanceState: Bundle?) {
24         super.onCreate(savedInstanceState)
25
26         // Inicializar la librería al abrir la app
27         try {
28             YoutubeDL.getInstance().init(this)
29         } catch (e: Exception) {
30             e.printStackTrace()
31         }
32         setContent {
```

MainActivity.kt ×

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
43 fun DownloaderUI(modifier: Modifier = Modifier) {
47
48     Column(modifier = modifier.padding(all = 16.dp)) {
49         TextField(
50             value = url,
51             onValueChange = { url = it },
52             label = { Text("Pega la URL aquí") },
53             modifier = Modifier.fillMaxWidth()
54         )
55         Button(
56             onClick = {
57                 if (url.isNotEmpty()) {
58                     // Lanzar proceso de descarga en segundo plano
59                     (context as? ComponentActivity)?.lifecycleScope?.launch(context = Dispatchers.IO) {
60                         try {
61                             val request = YoutubeDLRequest(url)
62                             YoutubeDL.getInstance().execute(request) { progress, etaInSeconds, line ->
63                                 // Aquí puedes actualizar una variable de estado para el progreso
64                             }
65                             withContext(Dispatchers.Main) {
66                                 Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
67                             }
68                         } catch (e: Exception) {
69                             withContext(Dispatchers.Main) {
70                                 Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
71                             }
72                         }
73                     }
74                 }
75             },
76             modifier = Modifier.padding(top = 16.dp).fillMaxWidth()
77         ) {
78             Text("Descargar")
79         }
80     }
81 }
```


MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
43     fun DownloaderUI(modifier: Modifier = Modifier) {
44         Column(modifier = modifier.padding(all = 16.dp)) {
45             value = url,
46             onChange = { url = it },
47             label = { Text("Pega la URL aquí") },
48             modifier = Modifier.fillMaxWidth()
49         }
50         Button(
51             onClick = {
52                 if (url.isNotEmpty()) {
53                     // Lanzar proceso de descarga en segundo plano
54                     (context as? ComponentActivity)?.lifecycleScope?.launch(context = Dispatchers.IO) {
55                         try {
56                             val downloadDir = context.getExternalFilesDir(type = Environment.DIRECTORY_DOWNLOADS)
57                             val request = YoutubeDLRequest(url)
58                             request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
59                             YoutubeDL.getInstance().execute(request) { progress, etaInSeconds, line ->
60                                 // Aquí puedes actualizar una variable de estado para el progreso
61                             }
62                             withContext(Dispatchers.Main) {
63                                 Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
64                             }
65                         } catch (e: Exception) {
66                             withContext(Dispatchers.Main) {
67                                 Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
68                             }
69                         }
70                     }
71                 }
72             }
73         ),
74         modifier = Modifier.padding(top = 16.dp).fillMaxWidth()
75     ) {
76         Text("Descargar")
77     }
78 }
79
80
81
82
83
```

```
context.getExternalFilesDir(Environment.DIRECTORY_DOWNLOADS)
```

Devuelve la carpeta de descargas privada de la app: algo como /sdcard/Android/data/com.baycodec.videodownloaderapp/files/Downloads.

Se eligió esta ruta porque no requiere el permiso `WRITE_EXTERNAL_STORAGE` en ninguna versión de Android — es el directorio propio de la app y el sistema siempre da acceso a él. La alternativa (carpeta pública Downloads) requeriría manejar MediaStore en Android 10+ y complicaría el código innecesariamente ahora.

```
request.addOption("-o", "...")
```

-o es la opción de yt-dlp que define el archivo de salida (output). Sin ella yt-dlp descarga al directorio de trabajo actual del proceso, que en Android no es accesible para el usuario.

```
%(title)s.%(ext)s
```

Es la plantilla de nombres de yt-dlp. `%(title)s` usa el título real del video y `%(ext)s` la extensión correcta (mp4, webm, etc.) que yt-dlp detecta automáticamente. Así el archivo queda con un nombre legible en vez de un hash.

MainActivity.kt ×

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
1 package com.baycode.videodownloaderapp
2
3 import android.os.Bundle
4 import android.os.Environment
5 import android.widget.Toast
6 import androidx.activity.ComponentActivity
7 import androidx.activity.compose.setContent
8 import androidx.compose.foundation.layout.*
9 import androidx.compose.material3.*
10 import androidx.compose.runtime.*
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.platform.LocalContext
13 import androidx.compose.ui.unit.dp
14 import androidx.lifecycle.LifecycleScope
15 import com.yausername.youtubedl_android.YouTubeDL
16 import com.yausername.youtubedl_android.YouTubeDLRequest
17 import kotlinx.coroutines.Dispatchers
18 import kotlinx.coroutines.launch
19 import kotlinx.coroutines.withContext
20
21 1 Usage
22 class MainActivity : ComponentActivity() {
23     override fun onCreate(savedInstanceState: Bundle?) {
24         super.onCreate(savedInstanceState)
25
26         // Inicializar la librería al abrir la app
27         try {
28             YouTubeDL.getInstance().init(this)
29         } catch (e: Exception) {
30             e.printStackTrace()
31         }
32
33         setContent {
34             MaterialTheme {
35                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
36                     DownloaderUI(Modifier.padding(paddingValues = innerPadding))
37                 }
38             }
39         }
40     }
41 }
```

MainActivity.kt ×

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
1 package com.baycode.videodownloaderapp
2
3 import android.os.Bundle
4 import android.os.Environment
5 import android.os.Handler
6 import android.os.Looper
7 import android.widget.Toast
8 import androidx.activity.ComponentActivity
9 import androidx.activity.compose.setContent
10 import androidx.compose.foundation.layout.*
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.*
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.platform.LocalContext
15 import androidx.compose.ui.unit.dp
16 import androidx.lifecycle.LifecycleScope
17 import com.yausername.youtubedl_android.YoutubeDL
18 import com.yausername.youtubedl_android.YoutubeDLRequest
19 import kotlinx.coroutines.Dispatchers
20 import kotlinx.coroutines.launch
21 import kotlinx.coroutines.withContext
22
```

1 Usage

```
23 class MainActivity : ComponentActivity() {
24     override fun onCreate(savedInstanceState: Bundle?) {
25         super.onCreate(savedInstanceState)
26
27         // Inicializar la librería al abrir la app
28         try {
29             YoutubeDL.getInstance().init(this)
30         } catch (e: Exception) {
31             e.printStackTrace()
32         }
33
34         setContent {
35             MaterialTheme {
36                 Scaffold(modifier = Modifier.fillMaxSize(), { innerPadding ->
```

MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
23     class MainActivity : ComponentActivity() {
24         override fun onCreate(savedInstanceState: Bundle?) {
34             setContent {
35                 MaterialTheme {
37                     DownloaderUI(Modifier.padding( paddingValues = innerPadding))
38                 }
39             }
40         }
41     }
42 }
43
44 1 Usage
45 @Composable
46 fun DownloaderUI(modifier: Modifier = Modifier) {
47     var url by remember { mutableStateOf( value = "") }
48     val context = LocalContext.current
49     val scope = rememberCoroutineScope()
50
51     Column(modifier = modifier.padding( all = 16.dp)) {
52         TextField(
53             value = url,
54             onValueChange = { url = it },
55             label = { Text("Pega la URL aquí") },
56             modifier = Modifier.fillMaxWidth()
57         )
58         Button(
59             onClick = {
60                 if (url.isNotEmpty()) {
61                     // Lanzar proceso de descarga en segundo plano
62                     (context as? ComponentActivity)?.lifecycleScope?.launch( context = Dispatchers.IO) {
63                         try {
64                             val downloadDir = context.getExternalFilesDir( type = Environment.DIRECTORY_DOWNLOADS)
65                             val request = YoutubeDLRequest(url)
66                             request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
67                             YoutubeDL.getInstance().execute(request) { progress, etaInSeconds, line ->
68                                 // Aquí puedes actualizar una variable de estado para el progreso
69                             }
70                         } catch (e: Exception) {
71                             // Manejar errores
72                         }
73                     }
74                 }
75             }
76         )
77     }
78 }
```

MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
23     class MainActivity : ComponentActivity() {
24         override fun onCreate(savedInstanceState: Bundle?) {
34             setContent {
39                 }
40             }
41         }
42     }
43
44     1 Usage
45     @Composable
46     fun DownloaderUI(modifier: Modifier = Modifier) {
47         var url by remember { mutableStateOf( value = "" ) }
48         var progress by remember { mutableFloatStateOf( value = 0f ) }
49         var isDownloading by remember { mutableStateOf( value = false ) }
50         val context = LocalContext.current
51         val mainHandler = remember { Handler(Looper.getMainLooper()) }
52
53         Column(modifier = modifier.padding( all = 16.dp )) {
54             TextField(
55                 value = url,
56                 onChange = { url = it },
57                 label = { Text("Pega la URL aquí") },
58                 modifier = Modifier.fillMaxWidth()
59             )
60             Button(
61                 onClick = {
62                     if (url.isNotEmpty()) {
63                         // Lanzar proceso de descarga en segundo plano
64                         (context as? ComponentActivity)?.lifecycleScope?.launch( context = Dispatchers.IO ) {
65                             try {
66                                 val downloadDir = context.getExternalFilesDir( type = Environment.DIRECTORY_DOWNLOADS )
67                                 val request = YoutubeDLRequest(url)
68                                 request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
69                                 YoutubeDL.getInstance().execute(request) { progress, etaInSeconds, line ->
70                                     // Aquí puedes actualizar una variable de estado para el progreso
71                                 }
72                             }
73                         }
74                     }
75                 }
76             )
77         }
78     }
79 }
```


MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
45     fun DownloaderUI(modifier: Modifier = Modifier) {
52         Column(modifier = modifier.padding(all = 16.dp)) {
54             value = url,
55             onChange = { url = it },
56             label = { Text("Pega la URL aquí") },
57             modifier = Modifier.fillMaxWidth()
58         }
59         Button(
60             onClick = {
61                 if (url.isNotEmpty()) {
62                     // Lanzar proceso de descarga en segundo plano
63                     (context as? ComponentActivity)?.lifecycleScope?.launch(context = Dispatchers.IO) {
64                         try {
65                             val downloadDir = context.getExternalFilesDir(type = Environment.DIRECTORY_DOWNLOADS)
66                             val request = YoutubeDLRequest(url)
67                             request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
68                             YoutubeDL.getInstance().execute(request) { progress, etaInSeconds, line ->
69                                 // Aquí puedes actualizar una variable de estado para el progreso
70                             }
71                             withContext(Dispatchers.Main) {
72                                 Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
73                             }
74                         } catch (e: Exception) {
75                             withContext(Dispatchers.Main) {
76                                 Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
77                             }
78                         }
79                     }
80                 }
81             },
82             modifier = Modifier.padding(top = 16.dp).fillMaxWidth()
83         ) {
84             Text("Descargar")
85         }
86     }
87 }
```

MainActivity.kt

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
45     fun DownloaderUI(modifier: Modifier = Modifier) {
52         Column(modifier = modifier.padding(all = 16.dp)) {
57             modifier = Modifier.fillMaxWidth()
58         }
59         Button(
60             onClick = {
61                 if (url.isNotEmpty()) {
62                     // Lanzar proceso de descarga en segundo plano
63                     (context as? ComponentActivity)?.lifecycleScope?.launch(context = Dispatchers.IO) {
64                         withContext(Dispatchers.Main) {
65                             isDownloading = true
66                             progress = 0f
67                         }
68                         try {
69                             val downloadDir = context.getExternalFilesDir(type = Environment.DIRECTORY_DOWNLOADS)
70                             val request = YoutubeDLRequest(url)
71                             request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
72                             YoutubeDL.getInstance().execute(request) { prog, _, _ ->
73                                 mainHandler.post { progress = prog }
74                             }
75                             withContext(Dispatchers.Main) {
76                                 Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
77                             }
78                         } catch (e: Exception) {
79                             withContext(Dispatchers.Main) {
80                                 Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
81                             }
82                         } finally {
83                             withContext(Dispatchers.Main) { isDownloading = false }
84                         }
85                     }
86                 }
87             },
88             modifier = Modifier.padding(top = 16.dp).fillMaxWidth()
89         ) {
90             Text("Descargar")
91         }
92     }
93 }
```

MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
45     fun DownloaderUI(modifier: Modifier = Modifier) {
52         Column(modifier = modifier.padding(all = 16.dp)) {
60             onClick = {
61                 if (url.isNotEmpty()) {
62                     // Lanzar proceso de descarga en segundo plano
63                     (context as? ComponentActivity)?.lifecycleScope?.launch(context = Dispatchers.IO) {
64                         withContext(Dispatchers.Main) {
65                             isDownloading = true
66                             progress = 0f
67                         }
68                         try {
69                             val downloadDir = context.getExternalFilesDir(type = Environment.DIRECTORY_DOWNLOADS)
70                             val request = YoutubeDLRequest(url)
71                             request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
72                             YoutubeDL.getInstance().execute(request) { prog, _, _ ->
73                                 mainHandler.post { progress = prog }
74                             }
75                             withContext(Dispatchers.Main) {
76                                 Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
77                             }
78                         } catch (e: Exception) {
79                             withContext(Dispatchers.Main) {
80                                 Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
81                             }
82                         } finally {
83                             withContext(Dispatchers.Main) { isDownloading = false }
84                         }
85                     }
86                 }
87             },
88             modifier = Modifier.padding(top = 16.dp).fillMaxWidth()
89         ) {
90             Text("Descargar")
91         }
92     }
93 }
```

MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
45     fun DownloaderUI(modifier: Modifier = Modifier) {
52         Column(modifier = modifier.padding(all = 16.dp)) {
60             onClick = {
63                 (context as? ComponentActivity)?.lifecycleScope?.launch(context = Dispatchers.IO) {
74                     }
75                 withContext(Dispatchers.Main) {
76                     Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
77                 }
78             } catch (e: Exception) {
79                 withContext(Dispatchers.Main) {
80                     Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
81                 }
82             } finally {
83                 withContext(Dispatchers.Main) { isDownloading = false }
84             }
85         }
86     },
87     modifier = Modifier.padding(top = 16.dp).fillMaxWidth()
88 ) {
89     Text("Descargar")
90 }
91
92
93 if (isDownloading) {
94     Spacer(modifier = Modifier.height(16.dp))
95     LinearProgressIndicator(
96         progress = { progress / 100f },
97         modifier = Modifier.fillMaxWidth()
98     )
99     Text(
100         text = "${progress.toInt()}%",
101         modifier = Modifier.padding(top = 4.dp)
102     )
103 }
104 }
105 }
```

`mutableFloatStateOf(0f)`

Versión optimizada de `mutableStateOf` para valores `Float` primitivos. `Compose` lo recompone solo cuando el valor cambia, sin `boxing/unboxing` innecesario.

`Handler(Looper.getMainLooper())`

El callback de `execute()` corre en el hilo de fondo (IO). `Compose` solo permite actualizar su estado desde el hilo principal. `Handler` con el `looper` principal es el puente: encola el `progress = prog` para que se ejecute en el hilo correcto.

`withContext(Dispatchers.Main) { isDownloading = true; progress = 0f }`

Resetea el estado antes de empezar, también desde el hilo principal. Si no se resetea, al hacer una segunda descarga el progreso arranca desde donde quedó la vez anterior.

`finally { isDownloading = false }`

`finally` se ejecuta siempre, haya éxito o error. Así el indicador desaparece en cualquier caso y no queda la barra atascada en pantalla.

`LinearProgressIndicator(progress = { progress / 100f })`

`yt-dlp` devuelve el progreso en `0..100`, pero `LinearProgressIndicator` espera `0f..1f`, por eso se divide entre 100. El indicador y el porcentaje solo se renderizan cuando `isDownloading` es `true`.

MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
1 Usage
44 @Composable
45 fun DownloaderUI(modifier: Modifier = Modifier) {
46     var url by remember { mutableStateOf( value = "" ) }
47     var progress by remember { mutableFloatStateOf( value = 0f ) }
48     var isDownloading by remember { mutableStateOf( value = false ) }
49     val context = LocalContext.current
50     val mainHandler = remember { Handler(Looper.getMainLooper()) }
51
52     Column(modifier = modifier.padding( all = 16.dp )) {
53         TextField(
54             value = url,
55             onValueChange = { url = it },
56             label = { Text("Pega la URL aquí") },
57             modifier = Modifier.fillMaxWidth()
58         )
59         Button(
60             onClick = {
61                 if (url.isNotEmpty()) {
62                     // Lanzar proceso de descarga en segundo plano
63                     (context as? ComponentActivity)?.lifecycleScope?.launch( context = Dispatchers.IO ) {
64                         withContext(Dispatchers.Main) {
65                             isDownloading = true
66                             progress = 0f
67                         }
68                         try {
69                             val downloadDir = context.getExternalFilesDir( type = Environment.DIRECTORY_DOWNLOADS )
70                             val request = YoutubeDLRequest(url)
71                             request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
72                             YoutubeDL.getInstance().execute(request) { prog, _, _ ->
73                                 mainHandler.post { progress = prog }
74                             }
75                             withContext(Dispatchers.Main) {
76                                 Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
77                             }
78                         } catch (e: Exception) {
79                             withContext(Dispatchers.Main) {
80                                 Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
81                             }
82                         } finally {
```


MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
45 fun DownloaderUI(modifier: Modifier = Modifier) {
46     var progress by remember { mutableFloatStateOf( value = 0f) }
47     var isDownloading by remember { mutableStateOf( value = false) }
48     val context = LocalContext.current
49     val mainHandler = remember { Handler(Looper.getMainLooper()) }
50
51     Column(modifier = modifier.padding( all = 16.dp)) {
52         TextField(
53             value = url,
54             onChange = { url = it },
55             label = { Text("Pega la URL aquí") },
56             modifier = Modifier.fillMaxWidth()
57         )
58         Button(
59             onClick = {
60                 if (url.isNotEmpty()) {
61                     // Lanzar proceso de descarga en segundo plano
62                     (context as? ComponentActivity)?.lifecycleScope?.launch( context = Dispatchers.IO) {
63                         withContext(Dispatchers.Main) {
64                             isDownloading = true
65                             progress = 0f
66                         }
67                         try {
68                             val downloadDir = context.getExternalFilesDir( type = Environment.DIRECTORY_DOWNLOADS)
69                             val request = YoutubeDLRequest(url)
70                             request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
71                             YoutubeDL.getInstance().execute(request) { prog, _, _ ->
72                                 mainHandler.post { progress = prog }
73                             }
74                             withContext(Dispatchers.Main) {
75                                 Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
76                             }
77                         } catch (e: Exception) {
78                             withContext(Dispatchers.Main) {
79                                 Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
80                             }
81                         } finally {
82                             withContext(Dispatchers.Main) { isDownloading = false }
83                         }
84                     }
85                 }
86             }
87         )
88     }
89 }
```


MainActivity.kt × build.gradle.kts (:app) AndroidManifest.xml settings.gradle.kts (VideoDownloaderApp) libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
45 fun DownloaderUI(modifier: Modifier = Modifier) {
46     var url by remember { mutableStateOf( value = "" ) }
47     var progress by remember { mutableFloatStateOf( value = 0f ) }
48     var isDownloading by remember { mutableStateOf( value = false ) }
49     val context = LocalContext.current
50     val mainHandler = remember { Handler(Looper.getMainLooper()) }
51
52     Column(modifier = modifier.padding( all = 16.dp )) {
53         TextField(
54             value = url,
55             onValueChange = { url = it },
56             label = { Text("Pega la URL aquí") },
57             modifier = Modifier.fillMaxWidth()
58         )
59         Button(
60             enabled = !isDownloading,
61             onClick = {
62                 if (url.isNotEmpty()) {
63                     (context as? ComponentActivity)?.lifecycleScope?.launch( context = Dispatchers.IO ) {
64                         withContext(Dispatchers.Main) {
65                             isDownloading = true
66                             progress = 0f
67                         }
68                         try {
69                             val downloadDir = context.getExternalFilesDir( type = Environment.DIRECTORY_DOWNLOADS )
70                             val request = YoutubeDLRequest(url)
71                             request.addOption("-o", "${downloadDir?.absolutePath}/%(title)s.%(ext)s")
72                             YoutubeDL.getInstance().execute(request) { prog, _, _ ->
73                                 mainHandler.post { progress = prog }
74                             }
75                             withContext(Dispatchers.Main) {
76                                 Toast.makeText(context, text = "Descarga completa", duration = Toast.LENGTH_SHORT).show()
77                             }
78                         } catch (e: Exception) {
79                             withContext(Dispatchers.Main) {
80                                 Toast.makeText(context, text = "Error: ${e.message}", duration = Toast.LENGTH_LONG).show()
81                             }
82                         } finally {
83                             withContext(Dispatchers.Main) { isDownloading = false }
84                         }
85                     }
86                 }
87             }
88         )
89     }
90 }
```

enabled = !isDownloading

Material3 maneja automáticamente el aspecto visual del botón deshabilitado (color gris, sin ripple) y bloquea los eventos de click.

Sin esto, el usuario puede pulsar varias veces seguidas lanzando descargas en paralelo que compiten por escribir el mismo archivo y saturan la red.

No hace falta añadir ningún if (isDownloading) return dentro del onClick — al deshabilitar el botón directamente, Android ya no dispara el evento, así que la guarda es innecesaria.

- ✓ — 1 — Dependencia youtubedl-android añadida al build
- ✓ — 2 — Permisos INTERNET y WRITE_EXTERNAL_STORAGE en el Manifest
- ✓ — 3 — Ruta de descarga con %(title)s.%(ext)s
- ✓ — 4 — Barra de progreso y porcentaje en la UI
- ✓ — 5 — Botón deshabilitado durante la descarga

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly. [Sync Now](#) [Ignore these changes](#)

```
45 fun DownloadUI(modifier: Modifier = Modifier) {
46     var url by remember { mutableStateOf("") }
47     var progress by remember { mutableFloatStateOf(0f) }
48     var isDownloading by remember { mutableStateOf(false) }
49     val context = LocalContext.current
50     val mainHandler = remember { Handler(Looper.getMainLooper()) }
51
52     Column(modifier = modifier.padding(all = 16.dp)) {
53         TextField(
54             value = url,
55             onChange = { url = it },
56             label = { Text("Pega la URL aquí") },
57             modifier = Modifier.fillMaxWidth()
58         )
59         Button(
60             enabled = !isDownloading,
61             onClick = {
62                 if (url.isNotEmpty()) {
63                     (context as? ComponentActivity)?.lifecycleScope?.launch(context = Dispatchers.IO) {
64                         withContext(Dispatchers.Main) {
65                             isDownloading = true
66                             progress = 0f
67                             try {
68                                 val downloadDir = context.getExternalFilesDir(Environment.DIRECTORY_DOWNLOADS)
```

Build VideoDownloaderApp: failed At 23/6/26, 23:30 with 1 error 6 sec, 956 ms

- New Minor Gradle Version Available
- Download Info
- Could Not Resolve com.github.yausername.youtubedl-android:library:0.17.2

Configuration cache state could not be cached: field '__runtimeDependenciesNavigationFiles__' of task ':app:processDebugNavigationResources' of type 'com.android.build.gradle.internal.tasks.ProcessNavigationResourcesTask'.

> Could not resolve all files for configuration ':app:debugRuntimeClasspath'.

> Could not find com.github.yausername.youtubedl-android:library:0.17.2.

Searched in the following locations:

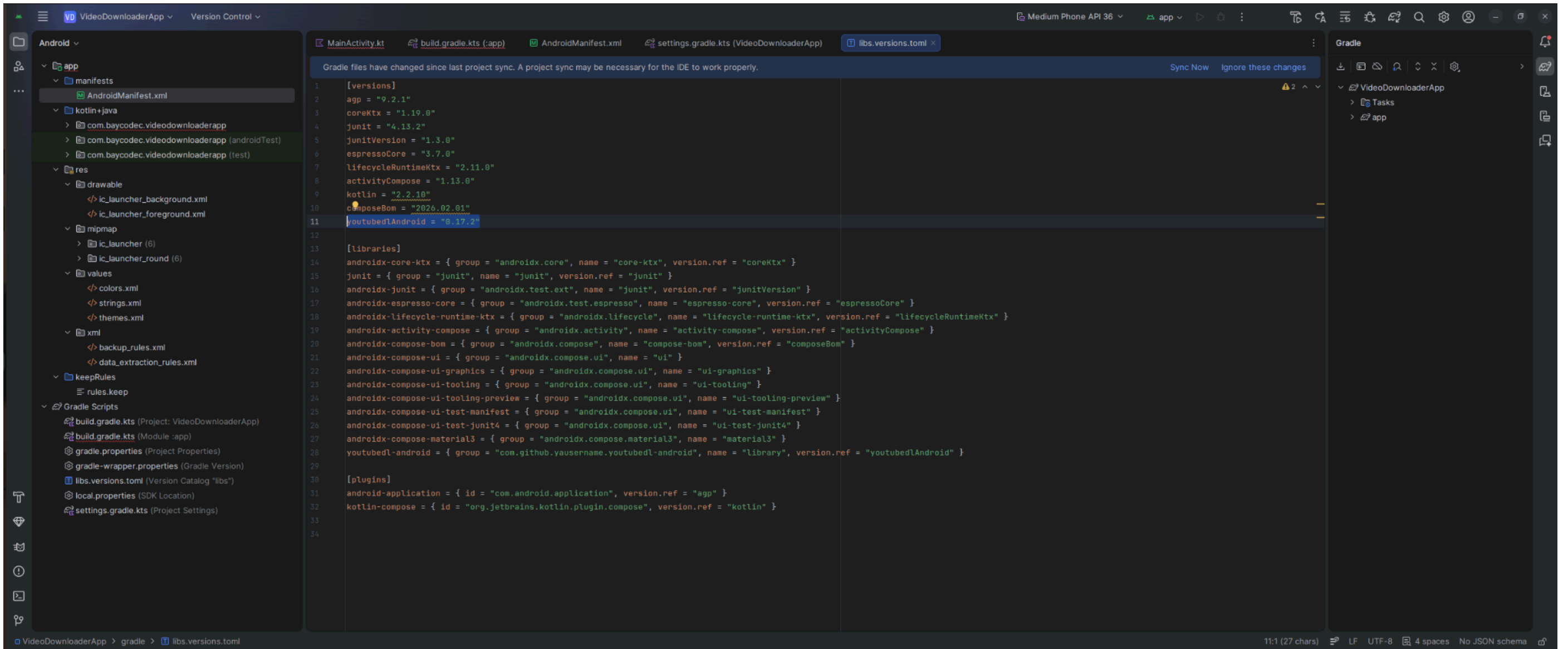
- https://dl.google.com/dl/android/maven2/com/github/yausername/youtubedl-android/library/0.17.2/library-0.17.2.pom
- https://repo.maven.apache.org/maven2/com/github/yausername/youtubedl-android/library/0.17.2/library-0.17.2.pom
- https://jitpack.io/com/github/yausername/youtubedl-android/library/0.17.2/library-0.17.2.pom

Required by:

- project ':app'

Possible solution:

- Declare repository providing the artifact, see the documentation at https://docs.gradle.org/current/userguide/declaring_repositories.html



Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

Sync Now

Ignore these changes

```
1  [versions]
2  app = "9.2.1"
3  coreKtx = "1.10.0"
4  junit = "4.13.2"
5  junitVersion = "1.3.0"
6  espressoCore = "3.7.0"
7  lifecycleRuntimeKtx = "2.11.0"
8  activityCompose = "1.13.0"
9  kotlin = "2.2.10"
10 composeBom = "2026.02.01"
11 youtubedlAndroid = "master-SNAPSHOT"
12
13 [libraries]
14 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
15 junit = { group = "junit", name = "junit", version.ref = "junit" }
16 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
17 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
18 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
19 androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
20 androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
21 androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
22 androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
23 androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
24 androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
25 androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
26 androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
27 androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
28 youtubedl-android = { group = "com.github.yausername.youtubedl-android", name = "library", version.ref = "youtubedlAndroid" }
29
30 [plugins]
31 android-application = { id = "com.android.application", version.ref = "app" }
32 kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
33
34
```

Gradle

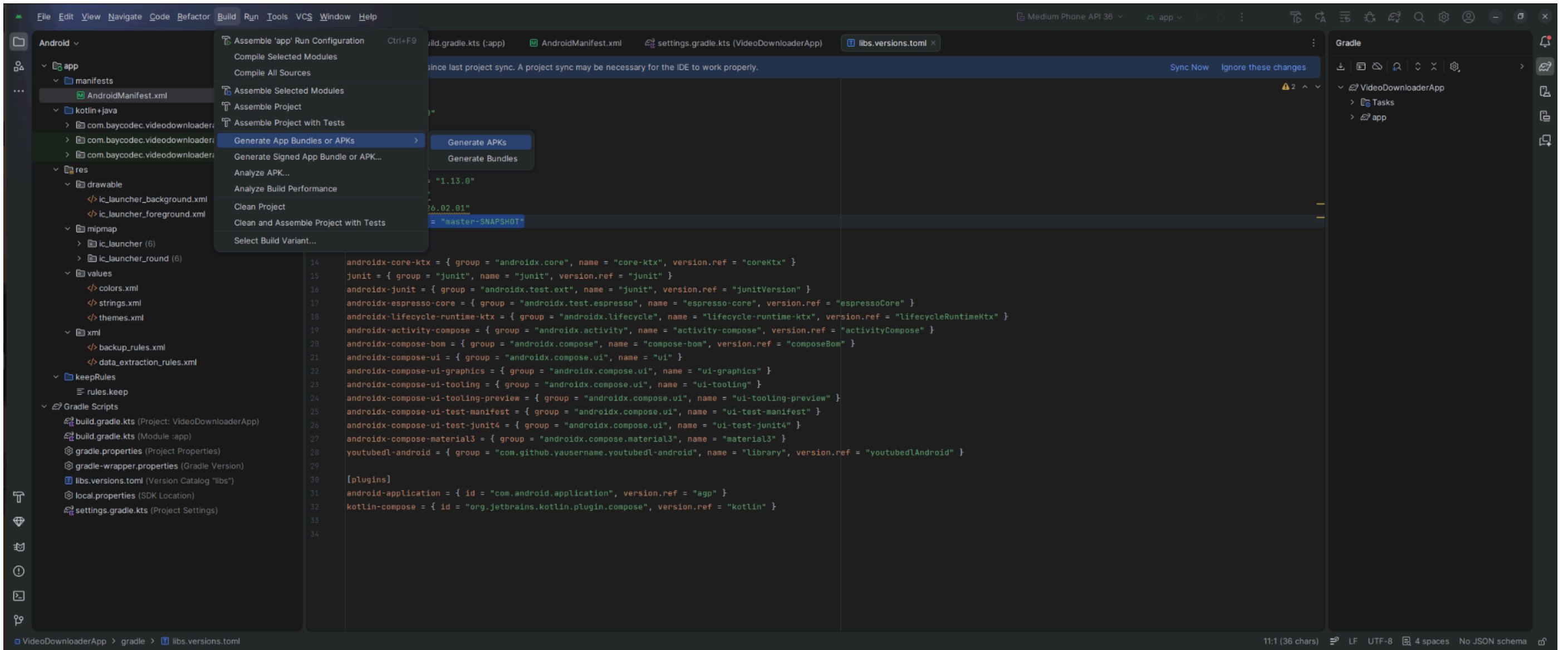
VideoDownloaderApp

Tasks

app

VideoDownloaderApp > gradle > libs.versions.toml

11:1 (36 chars) LF UTF-8 4 spaces No JSON schema



Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

Sync Now

Ignore these changes

```
1  [versions]
2  app = "9.2.1"
3  coreKtx = "1.10.0"
4  junit = "4.13.2"
5  junitVersion = "1.3.0"
6  espressoCore = "3.7.0"
7  lifecycleRuntimeKtx = "2.11.0"
8  activityCompose = "1.13.0"
9  kotlin = "2.2.10"
10 composeBom = "2026.02.01"
11 youtubedlAndroid = "master-SNAPSHOT"
12
13 [libraries]
14 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
15 junit = { group = "junit", name = "junit", version.ref = "junit" }
16 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
17 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
18 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
19 androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
20 androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
21 androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
22 androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
23 androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
24 androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
25 androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
26 androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
27 androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
28 youtubedl-android = { group = "com.github.yausername.youtubedl-android", name = "library", version.ref = "youtubedlAndroid" }
29
30 [plugins]
31 android-application = { id = "com.android.application", version.ref = "app" }
32 kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
33
34
```

Gradle

VideoDownloaderApp

Tasks

app

VideoDownloaderApp > gradle > libs.versions.toml

11:1 (36 chars) LF UTF-8 4 spaces No JSON schema

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

Sync Now

Ignore these changes

```
1  [versions]
2  app = "9.2.1"
3  coreKtx = "1.10.0"
4  junit = "4.13.2"
5  junitVersion = "1.3.0"
6  espressoCore = "3.7.0"
7  lifecycleRuntimeKtx = "2.11.0"
8  activityCompose = "1.13.0"
9  kotlin = "2.2.10"
10 composeBom = "2026.02.01"
11 youtubedlAndroid = "8.14.0"
12
13 [libraries]
14 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
15 junit = { group = "junit", name = "junit", version.ref = "junit" }
16 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
17 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
18 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
19 androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
20 androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
21 androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
22 androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
23 androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
24 androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
25 androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
26 androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
27 androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
28 youtubedl-android = { group = "com.github.yausername.youtubedl-android", name = "library", version.ref = "youtubedlAndroid" }
29
30 [plugins]
31 android-application = { id = "com.android.application", version.ref = "app" }
32 kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
33
34
```

Gradle

VideoDownloaderApp

Tasks

app

VideoDownloaderApp > gradle > libs.versions.toml

11:1 (27 chars) LF UTF-8 4 spaces No JSON schema

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

MainActivity.kt

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

Sync Now

Ignore these changes

```
1  [versions]
2  app = "9.2.1"
3  coreKtx = "1.10.0"
4  junit = "4.13.2"
5  junitVersion = "1.3.0"
6  espressoCore = "3.7.0"
7  lifecycleRuntimeKtx = "2.11.0"
8  activityCompose = "1.13.0"
9  kotlin = "2.2.10"
10 composeBom = "2026.02.01"
11 youtubedlAndroid = "8.14.0"
12
13 [libraries]
14 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
15 junit = { group = "junit", name = "junit", version.ref = "junit" }
16 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
17 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
18 androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
19 androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
20 androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
21 androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
22 androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
23 androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
24 androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
25 androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
```

Gradle

VideoDownloaderApp

Tasks

app

Build

Sync

Build Output

Build VideoDownloaderApp: failed At 23/6/26, 23:47 with 2 errors

30 sec, 439 ms

New Minor Gradle Version Available

Download info

:app:checkDebugAarMetadata 1 error

1 sec, 36 ms

3 issues were found when checking AAR metadata:

3 issues were found when checking AAR metadata:

1. Dependency 'androidx.core:core-ktx:1.10.0' requires libraries and applications that depend on it to compile against version 37 or later of the Android APIs.

:app is currently compiled against android-36.1.

Recommended action: Update this project to use a newer compileSdk of 37.

Note that updating a library or application's compileSdk (which allows newer APIs to be used) can be done separately from updating targetSdk (which sets the app to the new runtime behavior) and

VideoDownloaderApp > gradle > libs.versions.toml

11:1 (27 chars) LF UTF-8 4 spaces No JSON schema

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycode.videodownloaderapp

com.baycode.videodownloaderapp (androidTest)

com.baycode.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

MainActivity.kt

build.gradle.kts (app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

Sync Now

Ignore these changes

```
1 plugins {
2     alias(libs.plugins.android.application)
3     alias(libs.plugins.kotlin.compose)
4 }
5
6 android {
7     namespace = "com.baycode.videodownloaderapp"
8     compileSdk {
9         version = release( version = 36 ) {
10             minorApiLevel = 1
11         }
12     }
13
14     defaultConfig {
15         applicationId = "com.baycode.videodownloaderapp"
16         minSdk = 24
17         targetSdk = 36
18         versionCode = 1
19         versionName = "1.0"
20
21         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
22     }
23
24     buildTypes {
25         release {
```

Gradle

VideoDownloaderApp

Tasks

app

Build

Sync

Build Output

Build VideoDownloaderApp: failed At 23/6/26, 23:47 with 2 errors

30 sec, 439 ms

New Minor Gradle Version Available

Download info

:app:checkDebugAarMetadata 1 error

1 sec, 36 ms

3 issues were found when checking AAR metadata:

3 issues were found when checking AAR metadata:

1. Dependency 'androidx.core:core-ktx:1.19.0' requires libraries and applications that depend on it to compile against version 37 or later of the Android APIs.

:app is currently compiled against android-36.1.

Recommended action: Update this project to use a newer compileSdk of 37.

Note that updating a library or application's compileSdk (which allows newer APIs to be used) can be done separately from updating targetSdk (which sets the app in to new runtime behavior) and

VideoDownloaderApp

app

build.gradle.kts

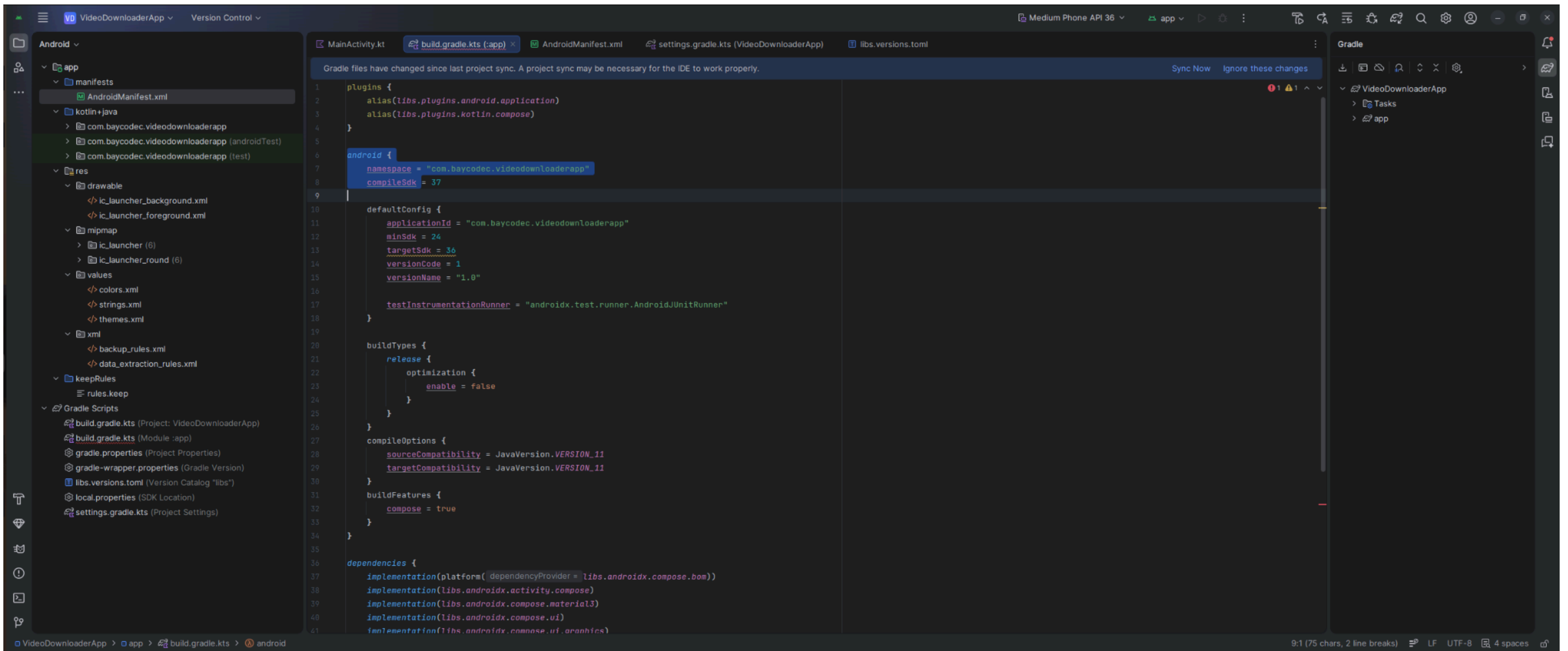
android

9:32 (108 chars, 3 line breaks)

LF

UTF-8

4 spaces



Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycode.videodownloaderapp

com.baycode.videodownloaderapp (androidTest)

com.baycode.videodownloaderapp (test)

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

1

plugins {

2

alias(libs.plugins.android.application)

3

alias(libs.plugins.kotlin.compose)

4

}

5

6

android {

7

namespace = "com.baycode.videodownloaderapp"

8

compileSdk = 37

9

10

defaultConfig {

11

applicationId = "com.baycode.videodownloaderapp"

12

minSdk = 24

13

targetSdk = 37

14

versionCode = 1

15

versionName = "1.0"

16

17

testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"

18

}

19

20

buildTypes {

21

release {

22

optimization {

23

enable = false

24

}

25

}

26

27

compileOptions {

28

sourceCompatibility = JavaVersion.VERSION_11

29

targetCompatibility = JavaVersion.VERSION_11

30

}

31

buildFeatures {

32

compose = true

33

}

34

}

35

36

dependencies {

37

implementation(platform(dependencyProvider = libs.androidx.compose.bom))

38

implementation(libs.androidx.activity.compose)

39

implementation(libs.androidx.compose.material3)

40

implementation(libs.androidx.compose.ui)

41

implementation(libs.androidx.compose.ui.graphics)

42

}

Gradle

VideoDownloaderApp

Tasks

app

VideoDownloaderApp

app

build.gradle.kts

android

13:1 (22 chars)

LF

UTF-8

4 spaces

```

..package com.baycode.videodownloaderapp

import android.os.Bundle
import android.os.Environment
import android.os.Handler
import android.os.Looper
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.lifecycle.lifecycleScope
import com.yausername.youtubedl_android.YoutubeDL
import com.yausername.youtubedl_android.YoutubeDLRequest
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Inicializar la librería al abrir la app
        try {
            YoutubeDL.getInstance().init(this)
        } catch (e: Exception) {
            e.printStackTrace()
        }

        setContent {
            MaterialTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding
->
                    DownloaderUI(Modifier.padding(innerPadding))
                }
            }
        }
    }
}

@Composable
fun DownloaderUI(modifier: Modifier = Modifier) {
    var url by remember { mutableStateOf("") }
    var progress by remember { mutableFloatStateOf(0f) }
    var isDownloading by remember { mutableStateOf(false) }
    val context = LocalContext.current
    val mainHandler = remember { Handler(Looper.getMainLooper()) }
}

Column(modifier = modifier.padding(16.dp)) {
    TextField(
        value = url,
        onValueChange = { url = it },
        label = { Text("Pega la URL aquí") },
        modifier = Modifier.fillMaxWidth()
    )
    Button(
        enabled = !isDownloading,
        onClick = {
            if (url.isNotEmpty()) {
                (context as?
ComponentActivity)?.lifecycleScope?.launch(Dispatchers.IO) {
                    withContext(Dispatchers.Main) {
                        isDownloading = true
                        progress = 0f
                    }
                    try {

```

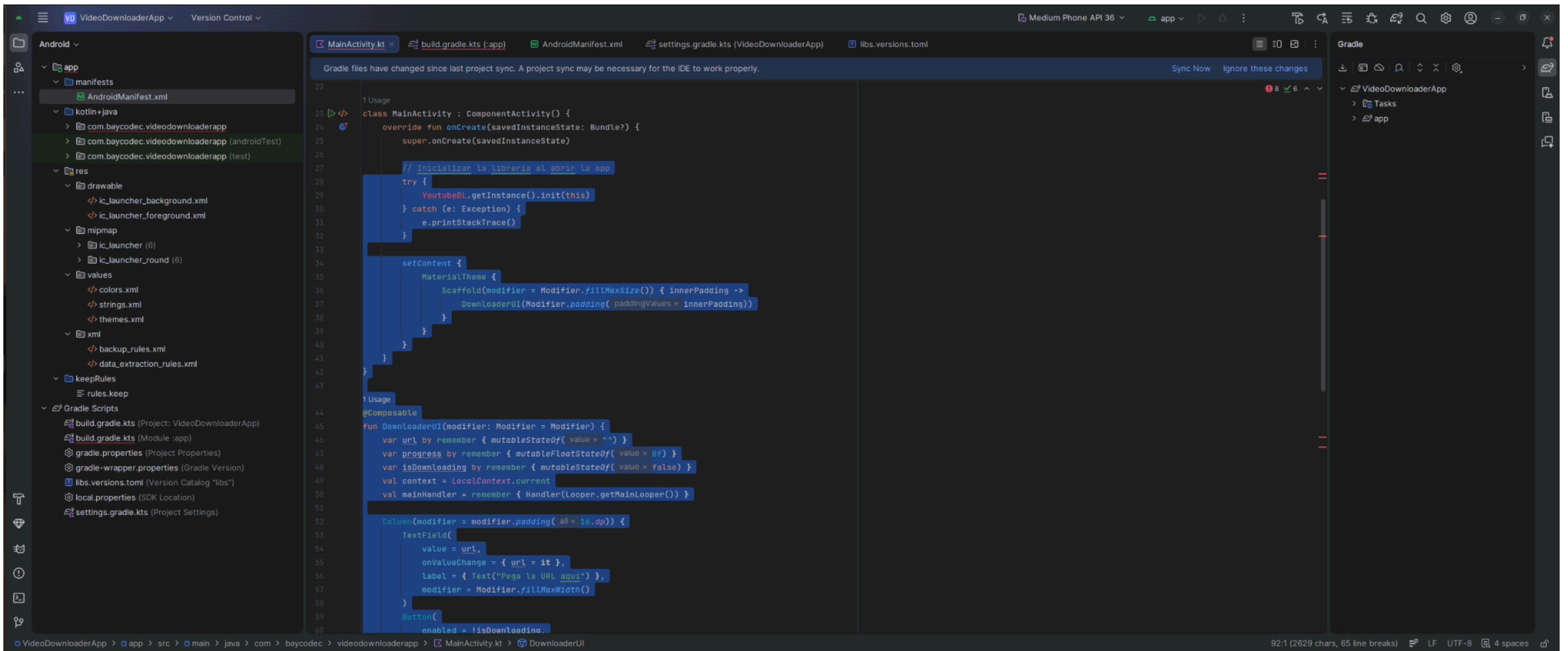
```

                        val downloadDir =
context.getExternalFilesDir(Environment.DIRECTORY_DOWNLOADS)

                        val request = YoutubeDLRequest(url)
                        request.addOption("-o",
"${downloadDir?.absolutePath}/${title}s.%(ext)s")
                        YoutubeDL.getInstance().execute(request) { prog,
-> _ ->
                            mainHandler.post { progress = prog }
                        }
                        withContext(Dispatchers.Main) {
                            Toast.makeText(context, "Descarga completa",
Toast.LENGTH_SHORT).show()
                        }
                    } catch (e: Exception) {
                        withContext(Dispatchers.Main) {
                            Toast.makeText(context, "Error: ${e.message}",
Toast.LENGTH_LONG).show()
                        }
                    } finally {
                        withContext(Dispatchers.Main) { isDownloading =
false }
                    }
                }
            },
            modifier = Modifier.padding(top = 16.dp).fillMaxWidth()
        ) {
            Text("Descargar")
        }

        if (isDownloading) {
            Spacer(modifier = Modifier.height(16.dp))
            LinearProgressIndicator(
                progress = { progress / 100f },
                modifier = Modifier.fillMaxWidth()
            )
            Text(
                text = "${progress.toInt()}%",
                modifier = Modifier.padding(top = 4.dp)
            )
        }
    }
}

```

```

1 27 - // Inicializar la librería al abrir la app
2 28 - try {
3 29 -     YoutubeDL.getInstance().init(this)
4 30 - } catch (e: Exception) {
5 31 -     e.printStackTrace()
6 27 + lifecycleScope.launch(Dispatchers.IO) {
7 28 +     try {
8 29 +         YoutubeDL.getInstance().init(application)
9 30 +         YoutubeDL.getInstance().updateYoutubeDL(application)
10 31 +     } catch (e: Exception) {
11 32 +         e.printStackTrace()
12 33 +     }
13 34 }
14 35
15 36 setContent {
16 ...
17 49 var progress by remember { mutableFloatStateOf(0f) }
18 50 var isDownloading by remember { mutableStateOf(false) }
19 51 val context = LocalContext.current
20 52 + val scope = rememberCoroutineScope()
21 53 val mainHandler = remember { Handler(Looper.getMainLooper()) }
22 54
23 55 Column(modifier = modifier.padding(16.dp)) {
24 ...
25 63         enabled = !isDownloading,
26 64         onClick = {
27 65             if (url.isNotEmpty()) {
28 63 -                 (context as? ComponentActivity)?.lifecycleScope?.launch(Dispatchers.IO) {
29 66 +                 scope.launch(Dispatchers.IO) {
30 67                     withContext(Dispatchers.Main) {
31 68                         isDownloading = true
32 69                         progress = 0f
33 ...
34 105                 )
35 106             }
36 107         }
37 105 -}
38 108 No newline at end of file
39 109 +}

```

updateYoutubeDL en onCreate

La librería youtubedl-android no incluye el binario de yt-dlp dentro del APK — lo descarga de internet la primera vez que se llama updateYoutubeDL. Sin esta llamada, no existe el ejecutable y ninguna descarga funciona. Se lanza en Dispatchers.IO para no bloquear el hilo principal mientras descarga el binario.

rememberCoroutineScope() en lugar de lifecycleScope del Activity

El problema anterior era que context as? ComponentActivity podía devolver null de forma silenciosa en Compose (el context puede estar envuelto en capas). Con rememberCoroutineScope() el scope está ligado directamente al composible, sin necesidad de castear nada, y es la forma correcta de lanzar coroutines desde un onClick en Compose.

Importante: la primera vez que abras la app tardará unos segundos antes de poder descargar — está descargando el binario de yt-dlp en segundo plano. Las siguientes veces arranca inmediato.


```

package com.baycode.videodownloaderapp

import android.os.Bundle
import android.os.Environment
import android.os.Handler
import android.os.Looper
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.lifecycleScope
import com.yausername.youtubedl_android.YoutubeDL
import com.yausername.youtubedl_android.YoutubeDLRequest
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

private const val TAG = "VideoDownloader"

private fun Throwable.fullTrace(): String {
    val sb = StringBuilder("${javaClass.name}: $message")
    stackTrace.take(5).forEach { sb.append("\n at $it") }
    cause?.let { sb.append("\nCausado por: ${it.fullTrace()}") }
    return sb.toString()
}

class MainActivity : ComponentActivity() {

    companion object {
        var initStatus by mutableStateOf("Iniciando yt-dlp...")
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        Log.d(TAG, "onCreate: llamando init()")
        var initOk = false
        try {
            YoutubeDL.getInstance().init(applicationContext)
            initOk = true
            initStatus = "Init OK ? descargando yt-dlp..."
            Log.d(TAG, "init() OK")
        } catch (e: Throwable) {
            initStatus = "Error init: ${e.fullTrace()}"
            Log.e(TAG, "init() fall💎", e)
        }

        if (initOk) {
            lifecycleScope.launch(Dispatchers.IO) {
                Log.d(TAG, "updateYoutubeDL: iniciando")
                try {
                    val status =
                        YoutubeDL.getInstance().updateYoutubeDL(applicationContext)
                    Log.d(TAG, "updateYoutubeDL resultado: $status")
                    withContext(Dispatchers.Main) { initStatus = "yt-dlp listo: $status" }
                } catch (e: Throwable) {
                    Log.e(TAG, "updateYoutubeDL fall💎", e)
                    withContext(Dispatchers.Main) { initStatus = "Error update: ${e.fullTrace()}" }
                }
            }
        }
    }
}

```

```

setContent {
    MaterialTheme {
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding
->
            DownloaderUI(Modifier.padding(innerPadding))
        }
    }
}

@Composable
fun DownloaderUI(modifier: Modifier = Modifier) {
    var url by remember { mutableStateOf("") }
    var progress by remember { mutableFloatStateOf(0f) }
    var isDownloading by remember { mutableStateOf(false) }
    var statusText by remember { mutableStateOf("") }
    val context = LocalContext.current
    val scope = rememberCoroutineScope()
    val mainHandler = remember { Handler(Looper.getMainLooper()) }

    Column(
        modifier = modifier
            .padding(16.dp)
            .verticalScroll(rememberScrollState())
    ) {
        Text(
            text = MainActivity.initStatus,
            fontSize = 12.sp,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )

        Spacer(modifier = Modifier.height(12.dp))

        TextField(
            value = url,
            onValueChange = { url = it },
            label = { Text("Pega la URL aqu💎") },
            modifier = Modifier.fillMaxWidth()
        )

        Button(
            enabled = !isDownloading,
            onClick = {
                if (url.isNotEmpty()) {
                    scope.launch(Dispatchers.IO) {
                        withContext(Dispatchers.Main) {
                            isDownloading = true
                            progress = 0f
                            statusText = "Iniciando..."
                        }
                    }
                    try {
                        val downloadDir =
                            context.getExternalFilesDir(Environment.DIRECTORY_DOWNLOADS)

                        Log.d(TAG, "downloadDir: ${downloadDir?.absolutePath}")
                        Log.d(TAG, "existe: ${downloadDir?.exists()}, escribible: ${downloadDir?.canWrite()}")
                        withContext(Dispatchers.Main) {
                            statusText = "Carpeta: ${downloadDir?.absolutePath}\nexiste=${downloadDir?.exists()} escribible=${downloadDir?.canWrite()}"
                        }

                        val request = YoutubeDLRequest(url)
                        request.addOption("-o", "${downloadDir?.absolutePath}/${title}s.%(ext)s")
                        Log.d(TAG, "ejecutando request para URL: $url")
                    }
                }
            }
        )
    }
}

```

```

        YoutubeDL.getInstance().execute(request) { prog,
eta, line ->
        Log.d(TAG, "progreso=$prog eta=$eta
linea=$line")
        mainHandler.post {
            progress = prog
            if (line.isNotBlank()) statusText = line
        }

        Log.d(TAG, "descarga completada")
        withContext(Dispatchers.Main) { statusText =
"Descarga completa" }

        } catch (e: Exception) {
            Log.e(TAG, "descarga falló", e)
            withContext(Dispatchers.Main) {
                statusText = "ERROR:\n${e.fullTrace()}"
            }
        } finally {
            withContext(Dispatchers.Main) { isDownloading =
false }
        }
    }
},
modifier = Modifier
    .padding(top = 16.dp)

```

```

        .fillMaxWidth()
    ) {
        Text("Descargar")
    }

    if (isDownloading) {
        Spacer(modifier = Modifier.height(16.dp))
        LinearProgressIndicator(
            progress = { progress / 100f },
            modifier = Modifier.fillMaxWidth()
        )
        Text(
            text = "${progress.toInt()}%",
            modifier = Modifier.padding(top = 4.dp)
        )
    }

    if (statusText.isNotEmpty()) {
        Spacer(modifier = Modifier.height(12.dp))
        Text(
            text = statusText,
            fontSize = 12.sp,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}
}
}

```

catch (e: Throwable) en vez de Exception — captura absolutamente cualquier error incluyendo Error, OutOfMemoryError, etc.

- applicationContext en vez de this — la librería espera el contexto de la aplicación, no el de la Activity
- if (initOk) — el update solo corre si el init tuvo éxito, así no se sobrescribe el error real

El problema: youtubedl-android incluye Python como un archivo .so binario que llega vía Git LFS. JitPack no descarga archivos de Git LFS, así que el AAR que descargamos está vacío de librerías nativas — el código Java está pero falta el Python.

Gradle recoge automáticamente los archivos en jniLibs/ y los empaqueta en el APK. Lo que tenemos ahora:

- libpython.so — la librería nativa de Python real (ELF) ✓
- libpython.zip.so — el entorno completo de Python en ZIP disfrazado de .so para que Android lo extraiga automáticamente ✓

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

jnilibs

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

Sync Now

Ignore these changes

```
18  android {
19
20      buildTypes {
21          release {
22              optimization {
23                  enable = false
24              }
25          }
26      }
27      compileOptions {
28          sourceCompatibility = JavaVersion.VERSION_11
29          targetCompatibility = JavaVersion.VERSION_11
30      }
31      buildFeatures {
32          compose = true
33      }
34  }
35
36  dependencies {
37      implementation(platform(dependencyProvider = libs.androidx.compose.bom))
38      implementation(libs.androidx.activity.compose)
39      implementation(libs.androidx.compose.material3)
40
41      // Extract Surround // se.ui)
42      implementation(libs.androidx.compose.ui.graphics)
43      implementation(libs.androidx.compose.ui.tooling.preview)
44      implementation(libs.androidx.core.ktx)
45      implementation(libs.androidx.lifecycle.runtime.ktx)
46      implementation(libs.youtubedl.android)
47      testImplementation(libs.junit)
48      androidTestImplementation(platform(dependencyProvider = libs.androidx.compose.bom))
49      androidTestImplementation(libs.androidx.compose.ui.test.junit4)
50      androidTestImplementation(libs.androidx.espresso.core)
51      androidTestImplementation(libs.androidx.junit)
52      debugImplementation(libs.androidx.compose.ui.test.manifest)
53      debugImplementation(libs.androidx.compose.ui.tooling)
54  }
```

Gradle

VideoDownloaderApp

Tasks

app

VideoDownloaderApp > app > build.gradle.kts > dependencies

39:52 (175 chars, 5 line breaks) LF UTF-8 4 spaces

Android

app

manifests

AndroidManifest.xml

kotlin+java

com.baycodec.videodownloaderapp

com.baycodec.videodownloaderapp (androidTest)

com.baycodec.videodownloaderapp (test)

jniLibs

res

drawable

ic_launcher_background.xml

ic_launcher_foreground.xml

mipmap

ic_launcher (6)

ic_launcher_round (6)

values

colors.xml

strings.xml

themes.xml

xml

backup_rules.xml

data_extraction_rules.xml

keepRules

rules.keep

Gradle Scripts

build.gradle.kts (Project: VideoDownloaderApp)

build.gradle.kts (Module :app)

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Version)

libs.versions.toml (Version Catalog "libs")

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

MainActivity.kt

build.gradle.kts (:app)

AndroidManifest.xml

settings.gradle.kts (VideoDownloaderApp)

libs.versions.toml

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

Sync Now

Ignore these changes

```
1  android {
2
3      buildTypes {
4          debug {
5              signingConfig = signingConfigs.getByName("debug")
6          }
7      }
8
9      compileOptions {
10         sourceCompatibility = JavaVersion.VERSION_11
11         targetCompatibility = JavaVersion.VERSION_11
12     }
13
14     buildFeatures {
15         compose = true
16     }
17
18     packaging {
19         jniLibs {
20             useLegacyPackaging = true
21         }
22     }
23 }
24
25 dependencies {
26     implementation(platform(dependencyProvider { libs.androidx.compose.bom }))
27     implementation(libs.androidx.activity.compose)
28     implementation(libs.androidx.compose.material3)
29     implementation(libs.androidx.compose.ui)
30     implementation(libs.androidx.compose.ui.graphics)
31     implementation(libs.androidx.compose.ui.tooling.preview)
32     implementation(libs.androidx.core.ktx)
33     implementation(libs.androidx.lifecycle.runtime.ktx)
34     implementation(libs.youtube.dl.android)
35     testImplementation(libs.junit)
36     androidTestImplementation(platform(dependencyProvider { libs.androidx.compose.bom }))
37     androidTestImplementation(libs.androidx.compose.ui.test.junit4)
38     androidTestImplementation(libs.androidx.espresso.core)
39     androidTestImplementation(libs.androidx.junit)
40     debugImplementation(libs.androidx.compose.ui.test.manifest)
41     debugImplementation(libs.androidx.compose.ui.tooling)
42 }
```

Gradle

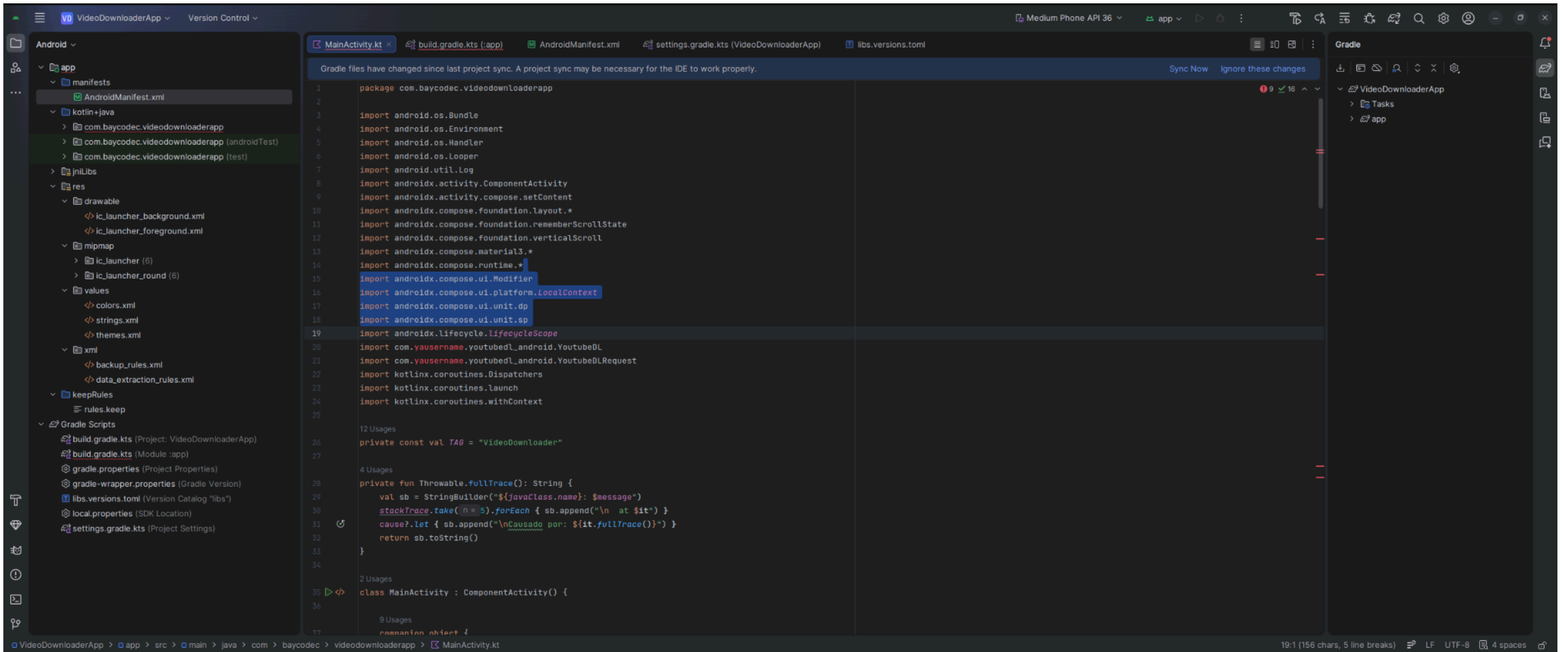
VideoDownloaderApp

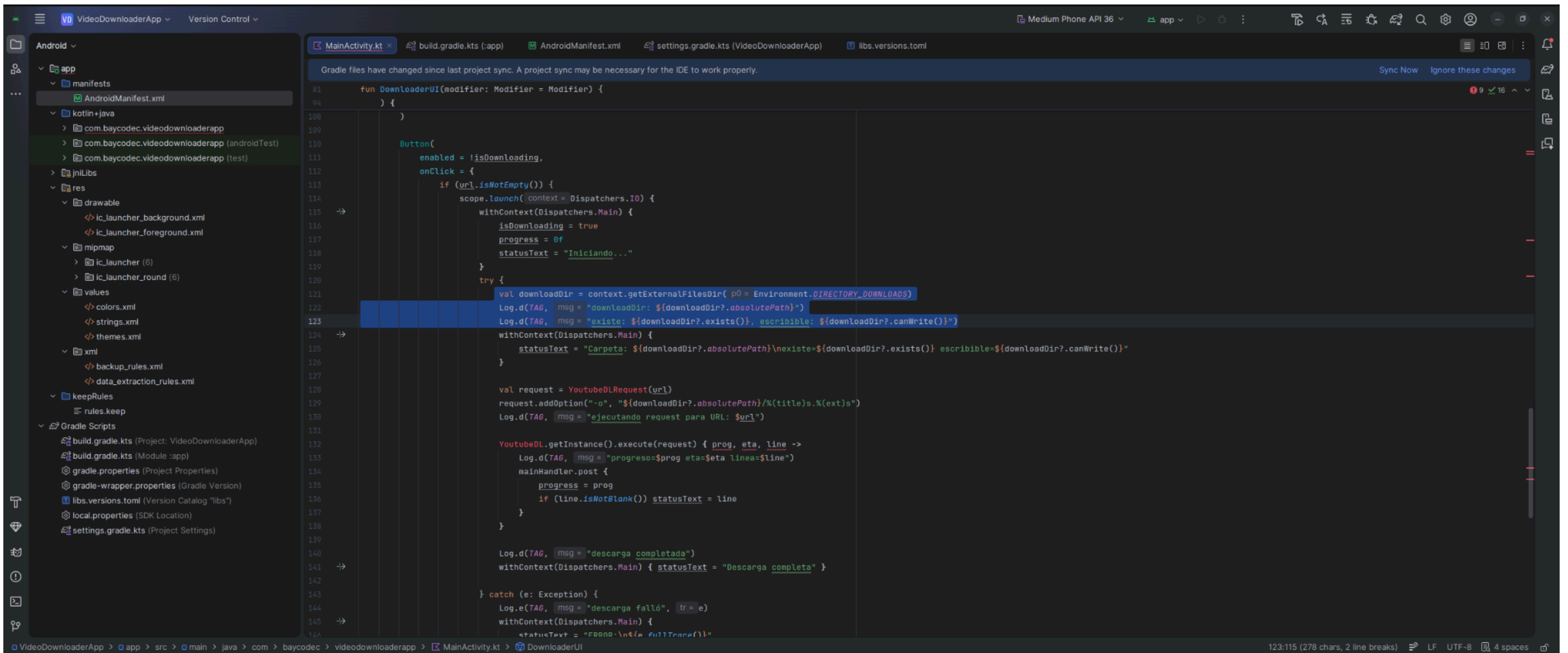
Tasks

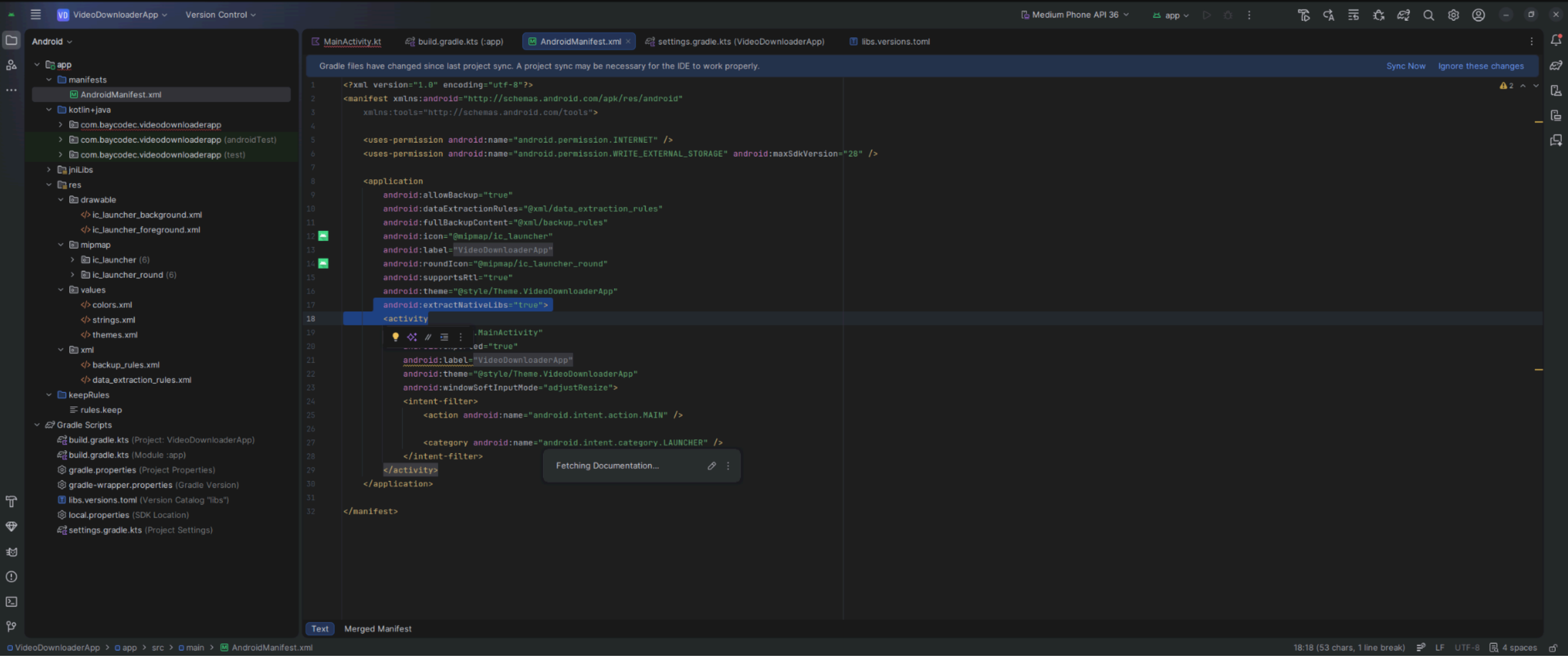
app

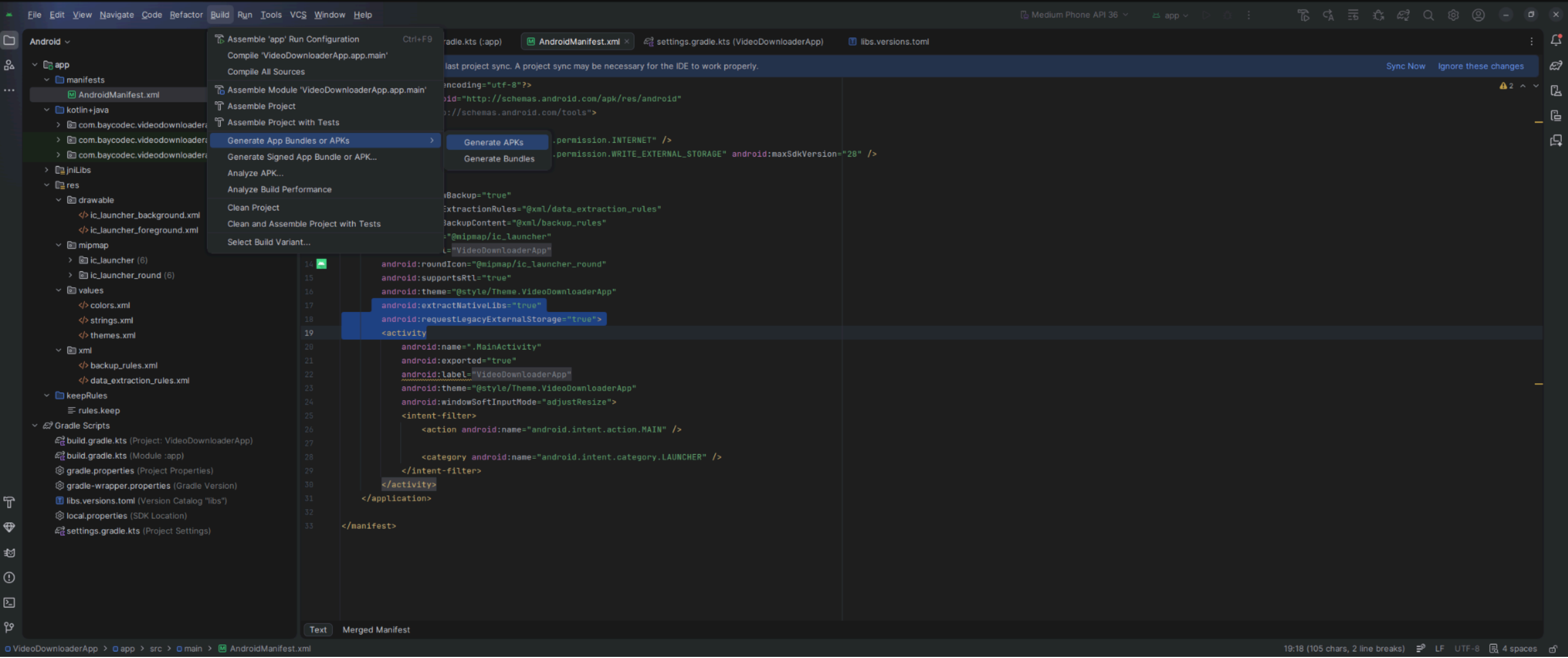
VideoDownloaderApp > app > build.gradle.kts > dependencies

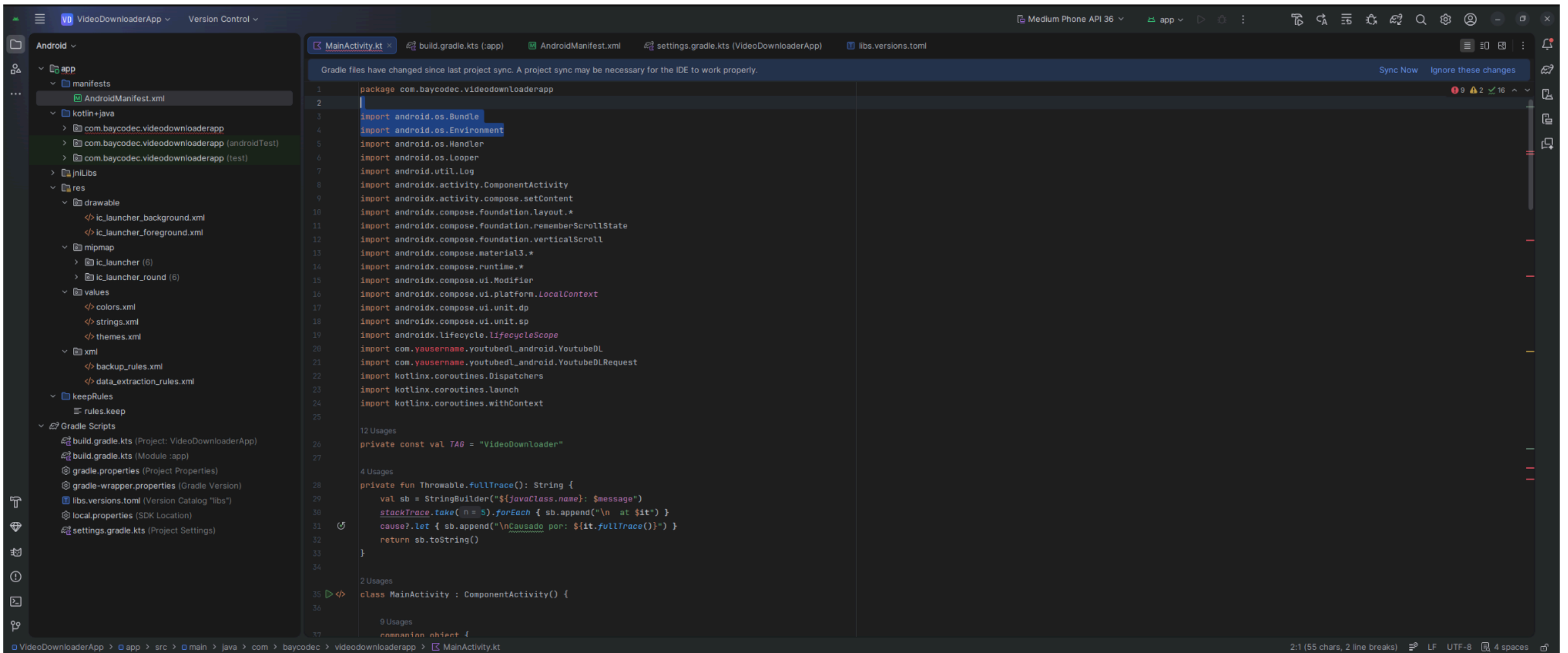
44:52 (175 chars, 5 line breaks) LF UTF-8 4 spaces

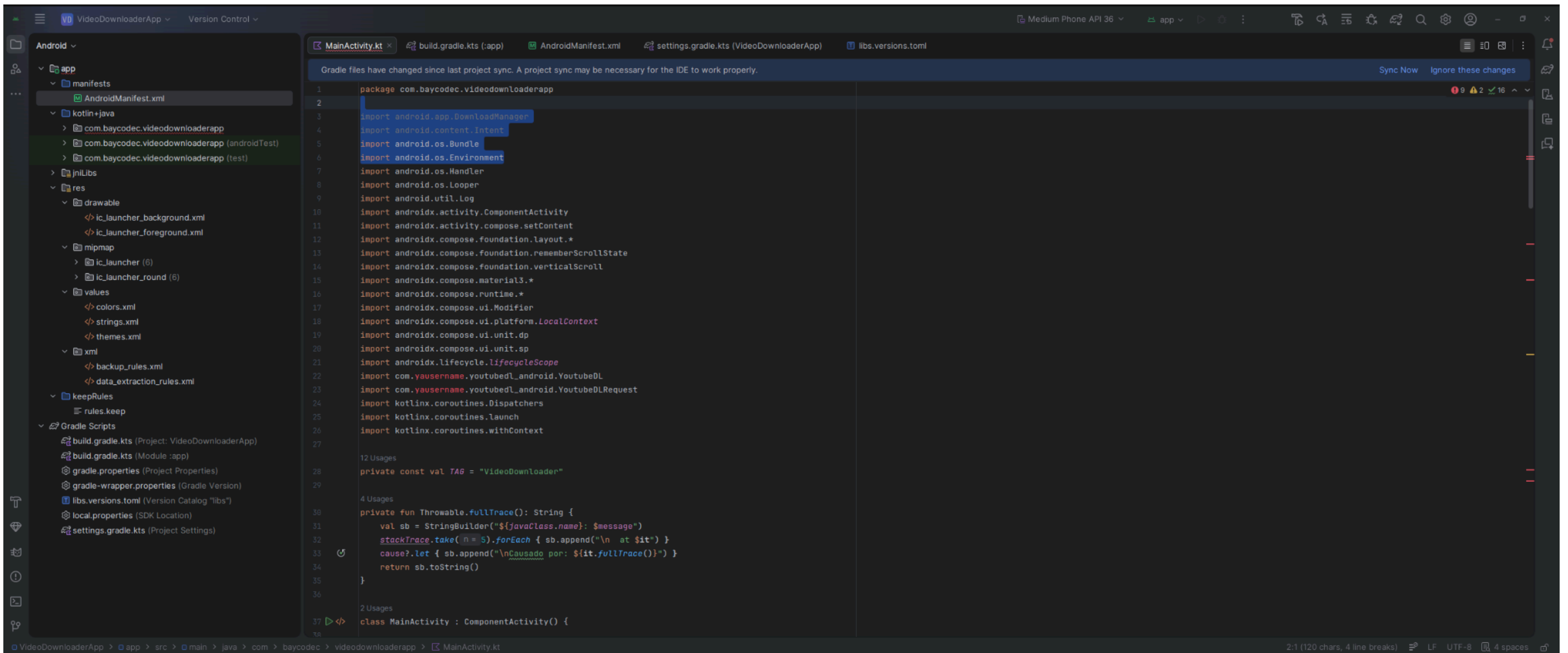


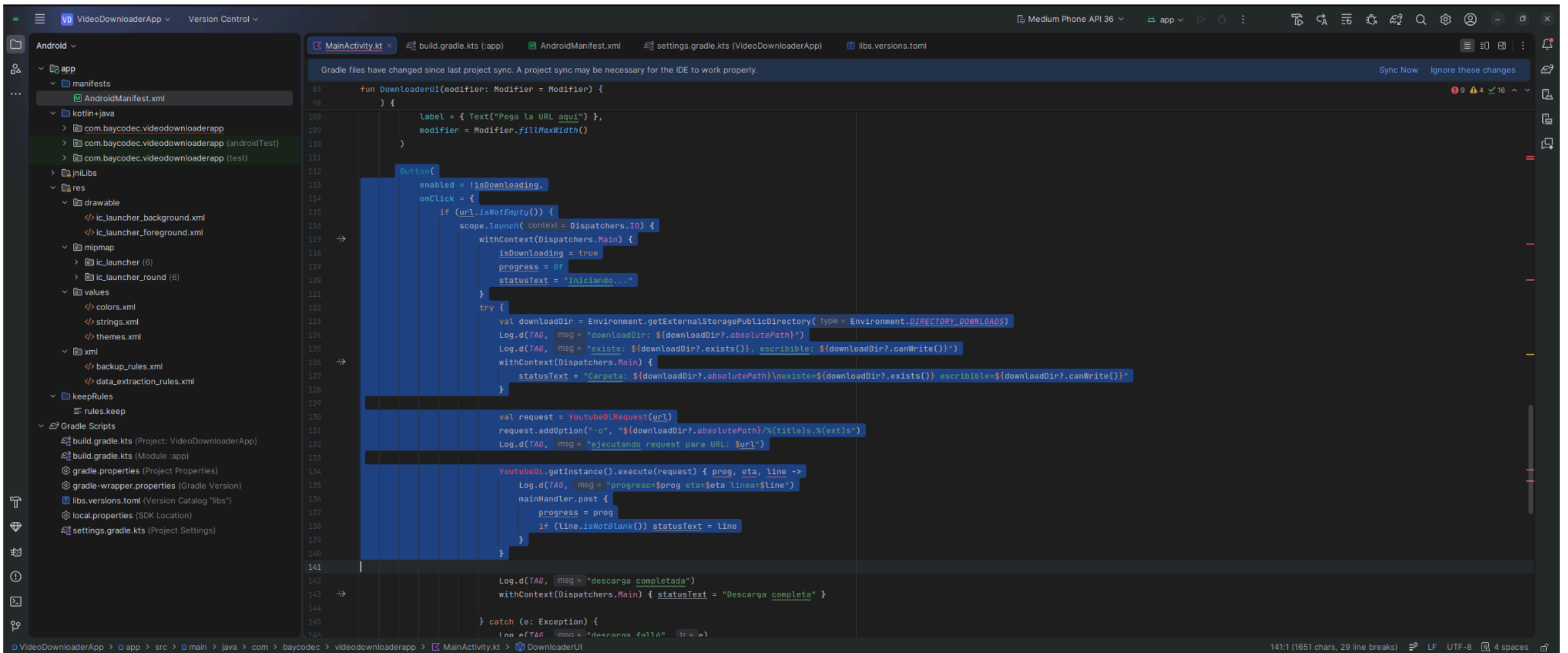


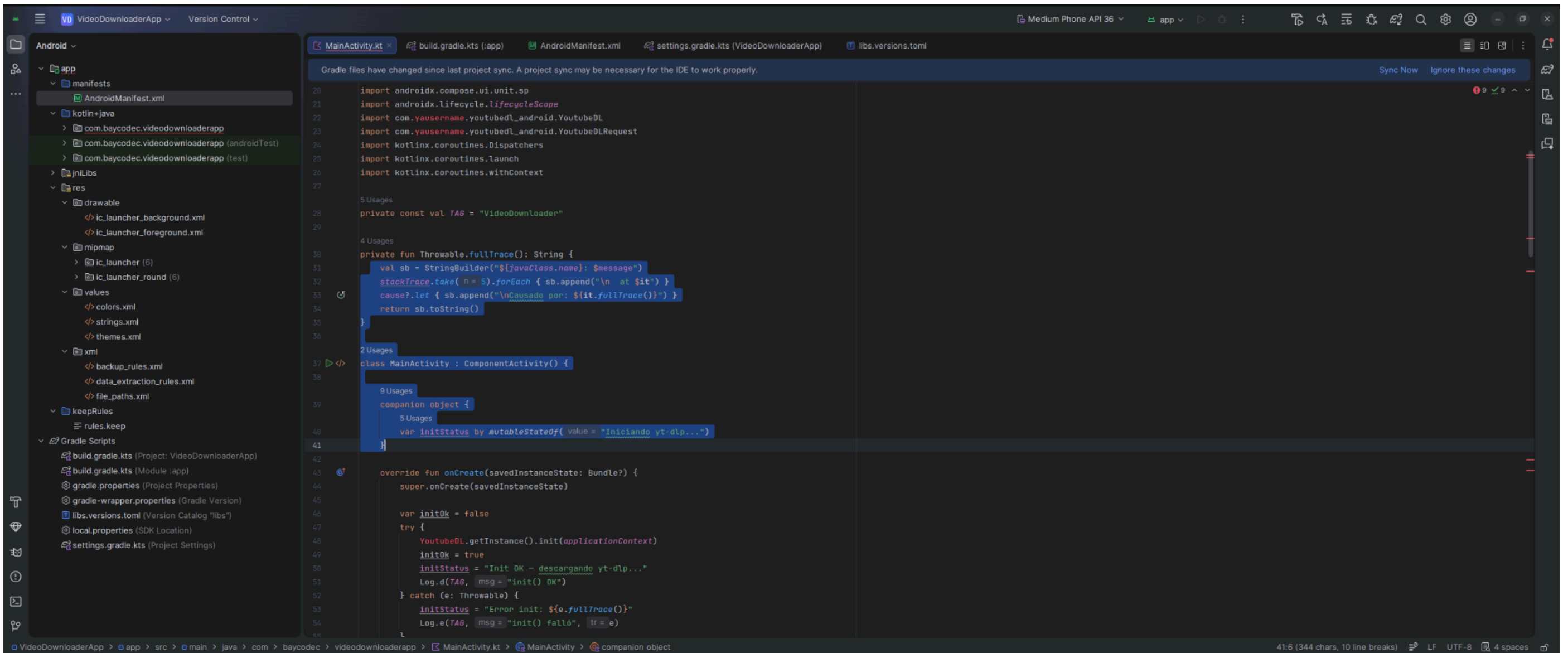












```
MainActivity.kt
1 package com.baycode.videodownloaderapp
2
3 import android.content.Intent
4 import android.os.Bundle
5 import android.os.Environment
6 import android.os.Handler
7 import android.os.Looper
8 import android.util.Log
9 import androidx.activity.ComponentActivity
10 import androidx.activity.compose.setContent
11 import androidx.compose.foundation.layout.*
12 import androidx.compose.foundation.rememberScrollState
13 import androidx.compose.foundation.verticalScroll
14 import androidx.compose.material3.*
15 import androidx.compose.runtime.*
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.platform.LocalContext
18 import androidx.compose.ui.unit.dp
19 import androidx.compose.ui.unit.sp
20 import androidx.core.content.FileProvider
21 import androidx.lifecycle.lifecycleScope
22 import com.yausername.youtubedl_android.YoutubeDL
23 import com.yausername.youtubedl_android.YoutubeDLRequest
24 import kotlinx.coroutines.Dispatchers
25 import kotlinx.coroutines.launch
26 import kotlinx.coroutines.withContext
27 import java.io.File
28
29 private const val TAG = "VideoDownloader"
30
31 private fun Throwable.fullTrace(): String {
32     val sb = StringBuilder("${javaClass.name}: $message")
33     stackTrace.take(5).forEach { sb.append("\n  at $it") }
34     cause?.let { sb.append("\nCausado por: ${it.fullTrace()}") }
35     return sb.toString()
36 }
37
38 class MainActivity : ComponentActivity() {
39     companion object {
40         var initStatus by mutableStateOf("Iniciando yt-dlp...")
41     }
42
43     override fun onCreate(savedInstanceState: Bundle?) {
44         super.onCreate(savedInstanceState)
45
46         var initOk = false
47         try {
48             YoutubeDL.getInstance().init(applicationContext)
49             initOk = true
50             initStatus = "Init OK – descargando yt-dlp..."
51         }
```

```

MainActivity.kt
49         YoutubeDL.getInstance().init(applicationContext)
50         initOk = true
51         initStatus = "Init OK – descargando yt-dlp..."
52         Log.d(TAG, "init() OK")
53     } catch (e: Throwable) {
54         initStatus = "Error init: ${e.fullTrace()}"
55         Log.e(TAG, "init() falló", e)
56     }
57
58     if (initOk) {
59         lifecycleScope.launch(Dispatchers.IO) {
60             try {
61                 val status = YoutubeDL.getInstance().updateYoutubeDL(applicationContext)
62                 withContext(Dispatchers.Main) { initStatus = "yt-dlp listo: $status" }
63             } catch (e: Throwable) {
64                 Log.e(TAG, "updateYoutubeDL falló", e)
65                 withContext(Dispatchers.Main) { initStatus = "Error update: ${e.fullTrace()}" }
66             }
67         }
68     }
69
70     setContent {
71         MaterialTheme {
72             Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
73                 DownloaderUI(Modifier.padding(innerPadding))
74             }
75         }
76     }
77 }
78
79
80 @Composable
81 fun DownloaderUI(modifier: Modifier = Modifier) {
82     var url by remember { mutableStateOf("") }
83     var progress by remember { mutableFloatStateOf(0f) }
84     var isDownloading by remember { mutableStateOf(false) }
85     var statusText by remember { mutableStateOf("") }
86     var lastDownloadedFile by remember { mutableStateOf<File?>(null) }
87     val context = LocalContext.current
88     val scope = rememberCoroutineScope()
89     val mainHandler = remember { Handler(Looper.getMainLooper()) }
90
91     Column(
92         modifier = modifier
93             .padding(16.dp)
94             .verticalScroll(rememberScrollState())
95     ) {
96         Text(
97             text = MainActivity.initStatus,
98             fontSize = 12.sp,
99             color = MaterialTheme.colorScheme.onSurfaceVariant

```



```

MainActivity.kt
94         .verticalScroll(rememberScrollState())
95     ) {
96         Text(
97             text = MainActivity.initStatus,
98             fontSize = 12.sp,
99             color = MaterialTheme.colorScheme.onSurfaceVariant
100        )
101
102        Spacer(modifier = Modifier.height(12.dp))
103
104        TextField(
105            value = url,
106            onValueChange = { url = it },
107            label = { Text("Pega la URL aquí") },
108            modifier = Modifier.fillMaxWidth()
109        )
110
111        Spacer(modifier = Modifier.height(12.dp))
112
113        Button(
114            enabled = !isDownloading,
115            onClick = {
116                if (url.isNotEmpty()) {
117                    scope.launch(Dispatchers.IO) {
118                        withContext(Dispatchers.Main) {
119                            isDownloading = true
120                            progress = 0f
121                            statusText = "Iniciando..."
122                            lastDownloadedFile = null
123                        }
124                    } try {
125                        val downloadDir = Environment.getExternalStoragePublicDirectory(
126                            Environment.DIRECTORY_DOWNLOADS
127                        )
128                        val request = YoutubeDLRequest(url)
129                        request.addOption("-o", "${downloadDir.absolutePath}/%(title)s.%(ext)s")
130                        request.addOption("-f", "bestvideo[ext=mp4]+bestaudio[ext=m4a]/best[ext=mp4]/best")
131                        request.addOption("--merge-output-format", "mp4")
132
133                        var detectedFile: File? = null
134
135                        YoutubeDL.getInstance().execute(request) { prog, _, line ->
136                            // detectar ruta del archivo de destino en la salida de yt-dlp
137                            if (line.contains("Destination:")) {
138                                val path = line.substringAfter("Destination:").trim()
139                                if (path.isNotEmpty()) detectedFile = File(path)
140                            }
141                            if (line.contains("Merging formats into")) {
142                                val path = line.substringAfter("\").substringBefore("\")
143                                if (path.isNotEmpty()) detectedFile = File(path)
144                            }

```

```

142         val path = line.substringAfter("\").substringBefore("\")
143         if (path.isNotEmpty()) detectedFile = File(path)
144     }
145     mainHandler.post {
146         progress = prog
147         if (line.isNotBlank()) statusText = line
148     }
149 }
150
151 // si no detectamos la ruta exacta, buscamos el MP4 más reciente
152 val finalFile = detectedFile?.takeIf { it.exists() }
153     ?: downloadDir.listFiles()
154         ?.filter { it.extension == "mp4" }
155         ?.maxByOrNull { it.lastModified() }
156
157 withContext(Dispatchers.Main) {
158     lastDownloadedFile = finalFile
159     statusText = if (finalFile != null)
160         "Descarga completa: ${finalFile.name}"
161     else
162         "Descarga completa"
163 }
164 } catch (e: Throwable) {
165     Log.e(TAG, "descarga falló", e)
166     withContext(Dispatchers.Main) {
167         statusText = "ERROR:\n${e.fullTrace()}"
168     }
169 } finally {
170     withContext(Dispatchers.Main) { isDownloading = false }
171 }
172     }
173 }
174 },
175 modifier = Modifier.fillMaxWidth()
176 ) {
177     Text("Descargar")
178 }
179
180 if (isDownloading) {
181     Spacer(modifier = Modifier.height(16.dp))
182     LinearProgressIndicator(
183         progress = { progress / 100f },
184         modifier = Modifier.fillMaxWidth()
185     )
186     Text(
187         text = "${progress.toInt()}%",
188         modifier = Modifier.padding(top = 4.dp)
189     )
190 }
191
192 lastDownloadedFile?.let { file ->

```



```

178     }
179
180     if (isDownloading) {
181         Spacer(modifier = Modifier.height(16.dp))
182         LinearProgressIndicator(
183             progress = { progress / 100f },
184             modifier = Modifier.fillMaxWidth()
185         )
186         Text(
187             text = "${progress.toInt()}%",
188             modifier = Modifier.padding(top = 4.dp)
189         )
190     }
191
192     lastDownloadedFile?.let { file ->
193         Spacer(modifier = Modifier.height(12.dp))
194         OutlinedButton(
195             onClick = {
196                 val uri = FileProvider.getUriForFile(
197                     context,
198                     "${context.packageName}.fileprovider",
199                     file
200                 )
201                 val intent = Intent(Intent.ACTION_VIEW).apply {
202                     setDataAndType(uri, "video/mp4")
203                     addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
204                     addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
205                 }
206                 context.startActivity(intent)
207             },
208             modifier = Modifier.fillMaxWidth()
209         ) {
210             Text("Abrir: ${file.name}")
211         }
212     }
213
214     if (statusText.isNotEmpty()) {
215         Spacer(modifier = Modifier.height(12.dp))
216         Text(
217             text = statusText,
218             fontSize = 12.sp,
219             color = MaterialTheme.colorScheme.onSurfaceVariant
220         )
221     }
222 }
223 }

```

yt-dlp listo: ALREADY_UP_TO_DATE

Pega la URL aquí

https://youtu.be/VcjzHMhBtf0?si=vpPVFnt8dqM5TP_s

Descargar

Abrir: Journey - Don't Stop Believin' (Escape Tour 1981: Live In Houston).mp4

Descarga completa: Journey - Don't Stop Believin' (Escape Tour 1981: Live In Houston).mp4

